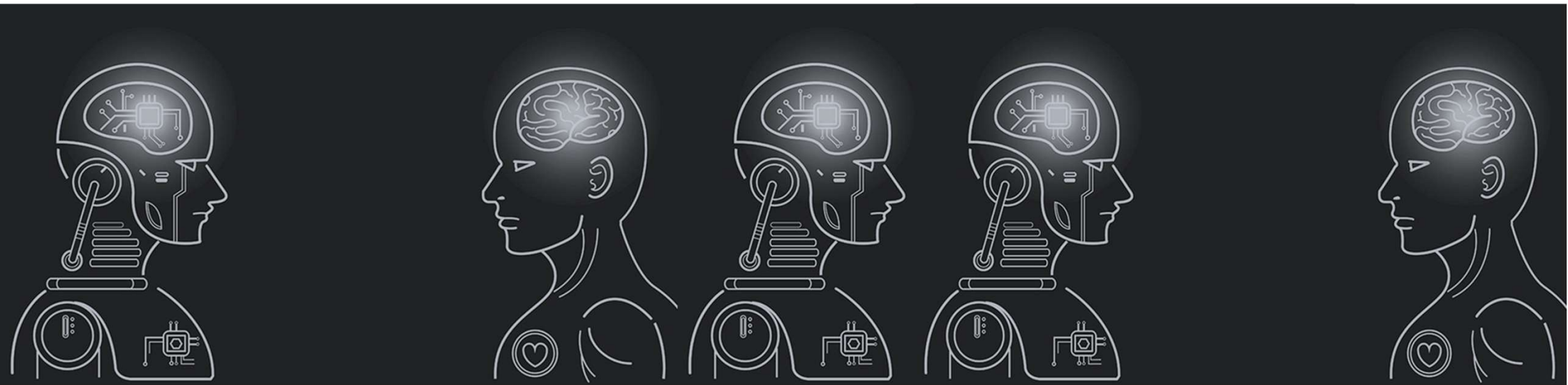


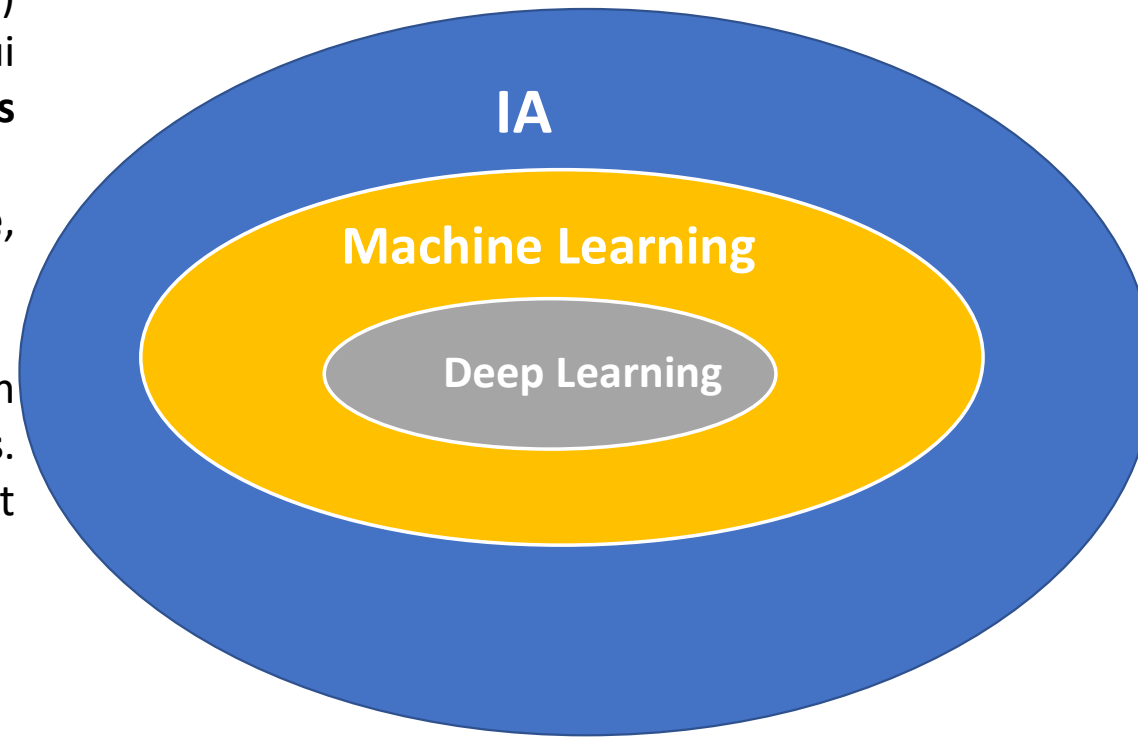
Machine Learning



Par Dr. CHAIBI Hasna
A.U 2025-2026

Qu'est-ce que le Machine Learning ?

- Le **Machine Learning** (ou **apprentissage automatique**) est une branche de l'**intelligence artificielle (IA)** qui permet aux ordinateurs **d'apprendre à partir des données** sans être explicitement programmés.
- Autrement dit, **la machine apprend de l'expérience**, comme un humain.
- Au lieu d'écrire toutes les règles manuellement, on **fournit à la machine des exemples**. Elle **analyse les données, découvre des modèles** et **fait des prédictions ou des décisions** toutes seules.



L'Intelligence Artificielle

- L'intelligence artificielle (IA) est « l'ensemble des théories et des techniques mises en œuvre en vue de réaliser des machines capables de simuler l'intelligence humaine »
- L'intelligence artificielle, c'est toute technologie informatique algorithmique qui permet de résoudre des problèmes complexes qu'on aurait cru réservés à l'intelligence humaine, en simulant des capacités humaines comme la perception et le raisonnement. - Cédric Villani -
- le Machine Learning, c'est une technique qui permet aux ordinateurs de réussir à faire tout ce genre de choses.

Machine Learning

□ Machine Learning, une technique au sein de L'IA

- Grâce au **Machine Learning**, nos ordinateurs sont capables de conduire des voitures et des avions de façon plus sûre que nous, ils peuvent diagnostiquer un patient (cancer, fracture) de façon plus fiable qu'un médecin.
- Le **Machine Learning** est aujourd'hui au cœur de nos systèmes de prise de décision : Banque, justice, sécurité, business, marketing. Les algorithmes de Google, Youtube, Facebook, Amazon, ou Netflix, fonctionnent tous grâce au Machine Learning.

Machine Learning

- **Le Machine Learning**, consiste à écrire un programme qui au début ne sait rien faire, mais qui va **apprendre** à faire quelque chose avec le **temps** et l'**expérience**... Un peu comme un être humain apprendrait à faire du vélo : au début on y arrive pas du tout, mais à force d'en faire et bien on y arrive de mieux en mieux, jusqu'au moment où on en fait super bien.
- Cet exemple colle hyper bien avec la définition du Machine Learning donnée par l'américain **Tom Mitchell** en **1998**. Selon lui, une machine apprend, lorsque sa **Performance P** à faire une **Tâche T** s'améliore grâce à une nouvelle **Expérience E**.
- **Tâche T** : Faire du vélo
- **Performance P** : Rouler droit, ne pas tomber
- **Expérience E** : chaque fois que l'on fait du vélo, y compris les fois où l'on tombe.

Machine Learning

- **L'apprentissage automatique**, ou *Machine Learning*, est un sous-ensemble de **l'intelligence artificielle** qui permet à un programme informatique d'effectuer une tâche pour laquelle il n'est pas programmé explicitement : il est programmé pour apprendre à la faire. **On donne au programme de nombreuses données et il apprend à partir de ces données.**

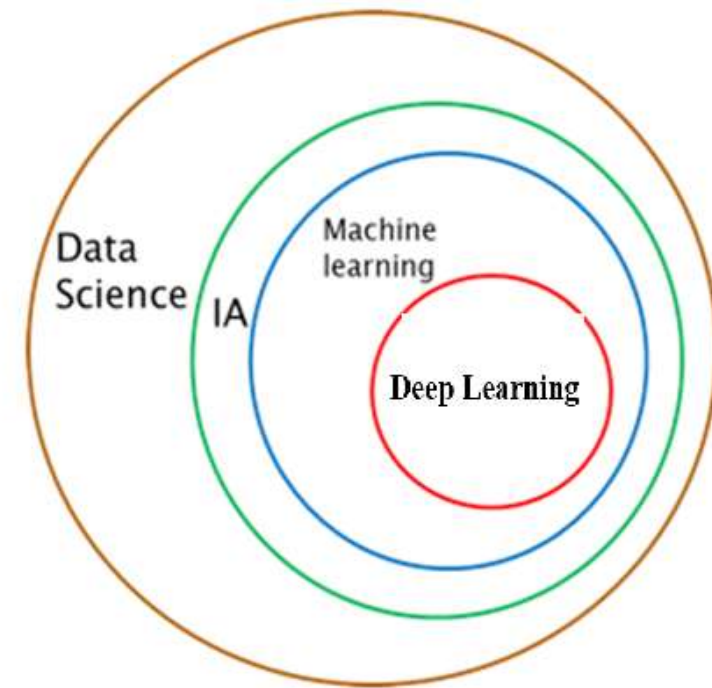
Machine Learning

□ Comment ça marche ?

- 1. Collecter des données**
- 2. Entraîner un modèle** à partir de ces données
- 3. Tester le modèle** sur de nouvelles données
- 4. Utiliser le modèle** pour faire des prédictions

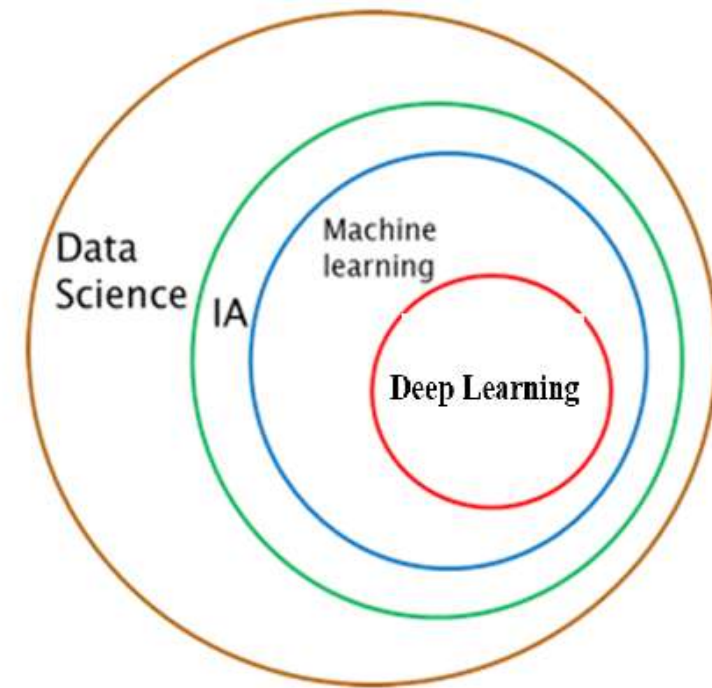
Deep Learning

- Le **Deep Learning**, qui est une discipline au sein même du Machine Learning, cherche à entraîner des modèles extrêmement **complexes** avec des **milliards de données (Big Data)** dans le but de surpasser les performances humaines.
- Le **Deep Learning** s'applique souvent sur des quantités de données beaucoup plus importantes que le Machine Learning. Il apprend de cette masse d'exemples et obtient dans certains cas de bien meilleurs résultats que les disciplines traditionnelles d'intelligence artificielle.
- Le **Machine Learning** et Le **Deep Learning** sont aujourd'hui des domaines en plein essor, ceci grâce à l'émergence des objets connectés (IoT) en 2007 (lancement de l'iPhone) qui fournissent aujourd'hui une grande quantité de données (**Big Data**) pour entraîner les modèles des **Data Scientists**.



Deep Learning

- Des **caméras, capteurs, réseaux sociaux** génèrent des données chaque seconde.
- Le ML et le DL progressent grâce à l'**explosion des données**.
- Dans le passé, **le manque de données** freinait l'entraînement des modèles.
- Aujourd'hui, **les données sont le carburant de l'IA**.
- L'IA et la **Data Science** sont deux disciplines qui sont utilisées conjointement, notamment pour mettre en place du **ML** ou du **DL**



Exemples d'applications du Machine Learning

□ Détection des spams (emails indésirables)

- **Exemple** : Gmail ou Outlook détectent automatiquement les e-mails indésirables.

Comment ça marche :

Le modèle est entraîné à partir de milliers d'exemples d'e-mails classés comme *spam* ou *non spam*.

Il apprend à reconnaître certains **mots-clés, liens suspects, ou comportements d'expéditeur**, et peut ensuite classer les nouveaux e-mails automatiquement.

Exemples d'applications du Machine Learning

❑ Recommandation de musique, films ou produits

- **Exemple :** Netflix, YouTube, Spotify, Amazon

Comment ça marche :

Le système apprend à partir de ton **historique d'écoute ou de visionnage**.

Il compare votre profil avec celui d'autres utilisateurs pour **vous recommander du contenu similaire** que vous pourrez aimer.

- ✓ C'est ce qu'on appelle un **système de recommandation**.



Exemples d'applications du Machine Learning

☐ Diagnostic médical assisté par IA

- **Exemple :** Détection de tumeurs dans des images radiologiques.

Comment ça marche :

Le modèle est entraîné avec des milliers d'images médicales (scanner, IRM, radios) étiquetées par des médecins.

Il apprend à **reconnaître les motifs** correspondant à certaines maladies.

- ✓ Cela aide les médecins à **détecter plus vite et avec précision** certaines pathologies.



Exemples d'applications du Machine Learning

☐ Voitures autonomes

- **Exemple :** Tesla, Waymo
Comment ça marche :



Les voitures utilisent des **caméras, capteurs et radars** pour observer l'environnement.

Le modèle de machine learning apprend à **identifier les objets** (piétons, feux, routes) et à **prendre des décisions** (freiner, tourner, accélérer).

- ✓ C'est une combinaison d'apprentissage supervisé, de vision par ordinateur et d'apprentissage par renforcement.

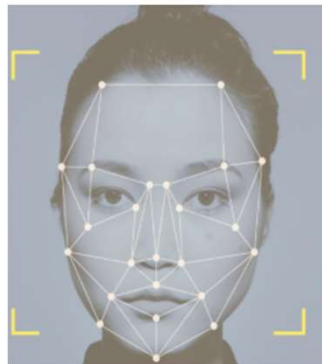
Exemples d'applications du Machine Learning

□ Reconnaissance d'images et de visages

- **Exemple** : Déverrouillage de téléphone avec reconnaissance faciale
Comment ça marche :

Le modèle apprend à partir de **milliers de photos de visages**.

Il reconnaît des **caractéristiques uniques** (distance entre les yeux, forme du nez, etc.) pour **identifier une personne**.



Exemples d'applications du Machine Learning

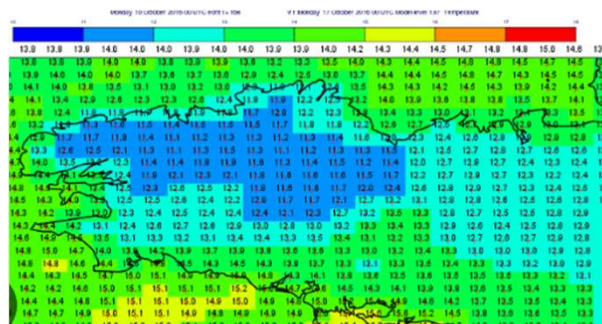
❑ Prédiction météorologique

- **Exemple : Météo sur smartphone**

Comment ça marche :

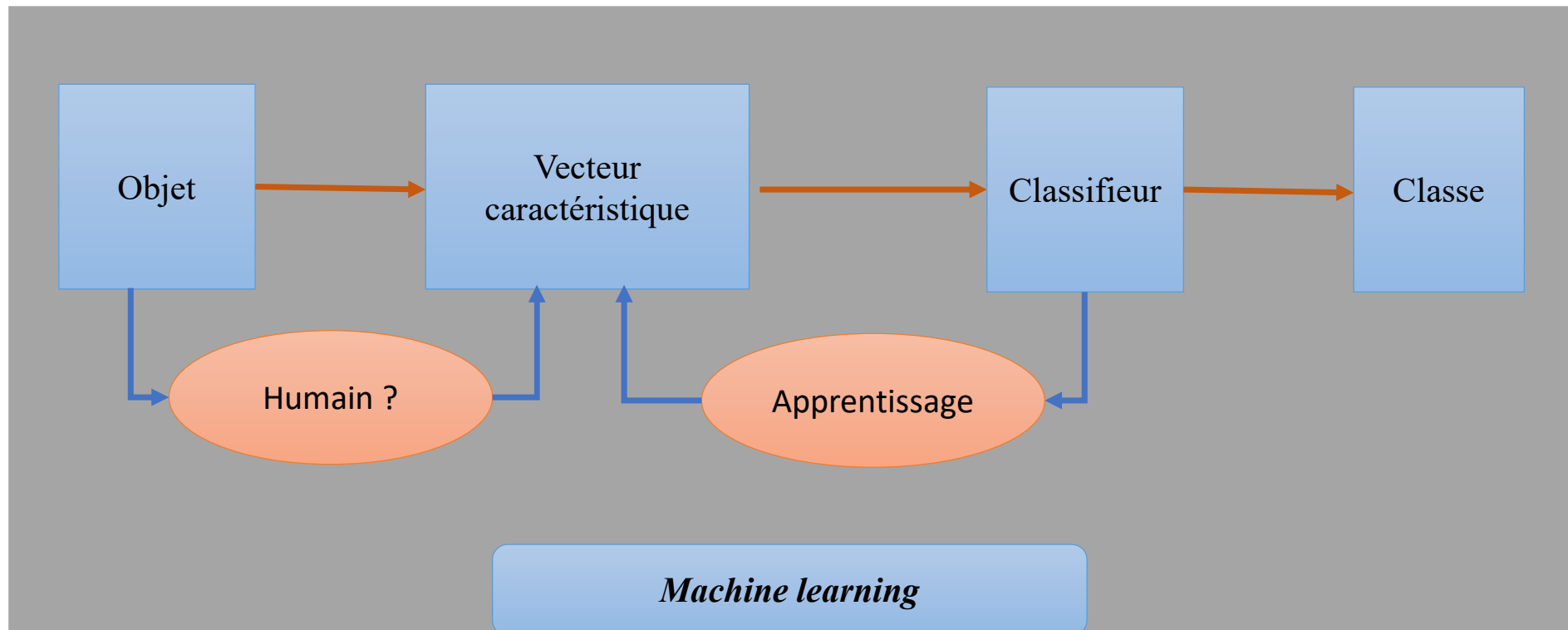
Les modèles analysent des **millions de données météorologiques** (température, humidité, pression...) pour **prédire le temps à venir**.

- ✓ Plus les données sont nombreuses, plus les prévisions sont précises.



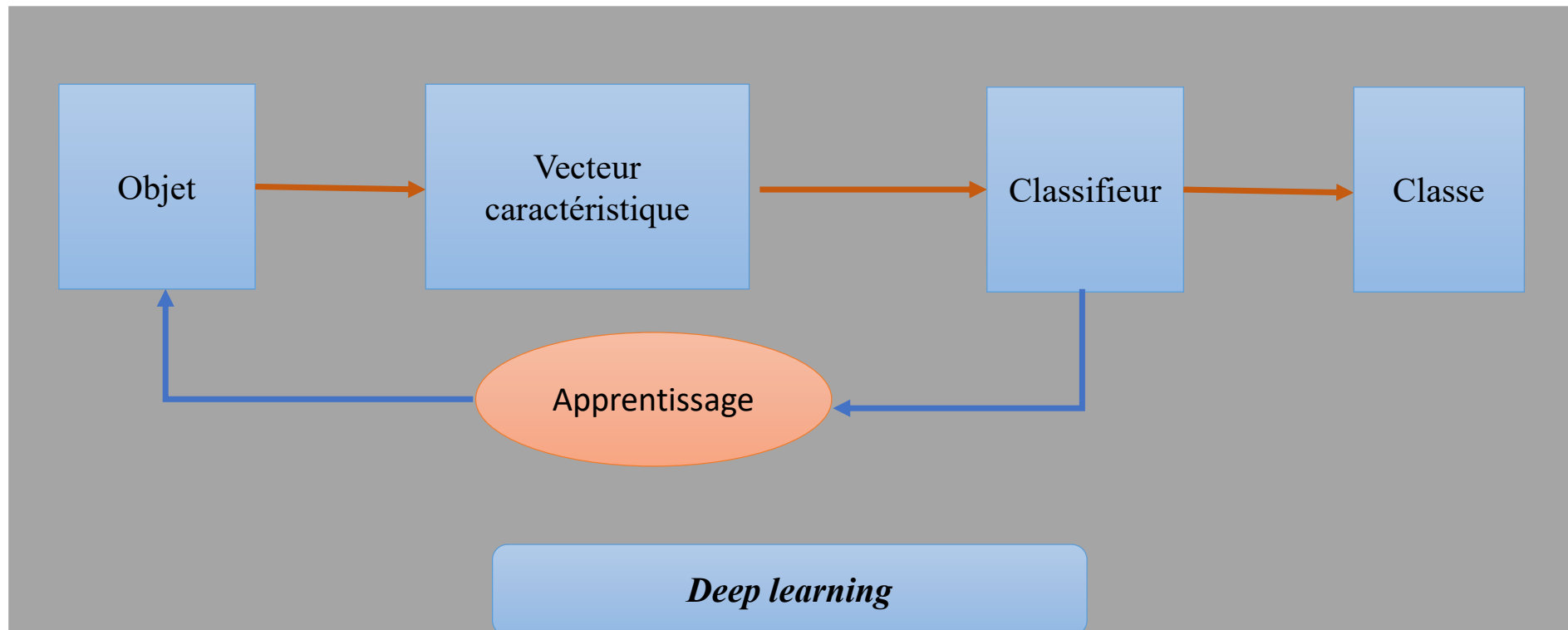
L'apprentissage profond -Deep-learning-

Machine learning Vs Deep learning



L'apprentissage profond -Deep-learning-

Machine learning Vs Deep learning



Types d'apprentissage

**Apprentissage
supervisé**

**Apprentissage non
supervisé**

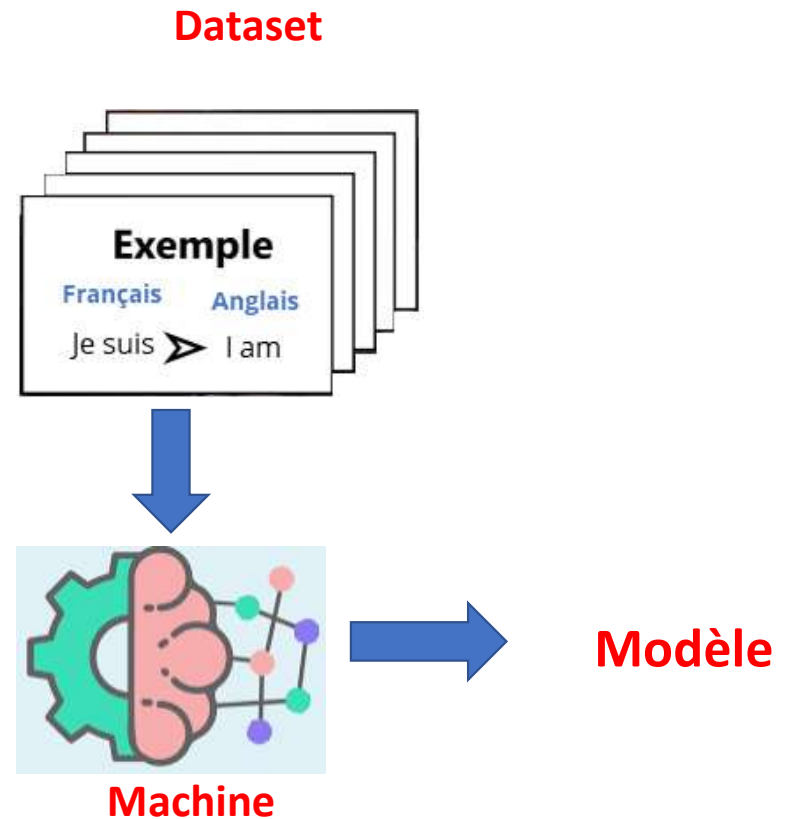
Exemple

Français Anglais
Je suis ➤ I am



Machine Learning

Types d'apprentissage



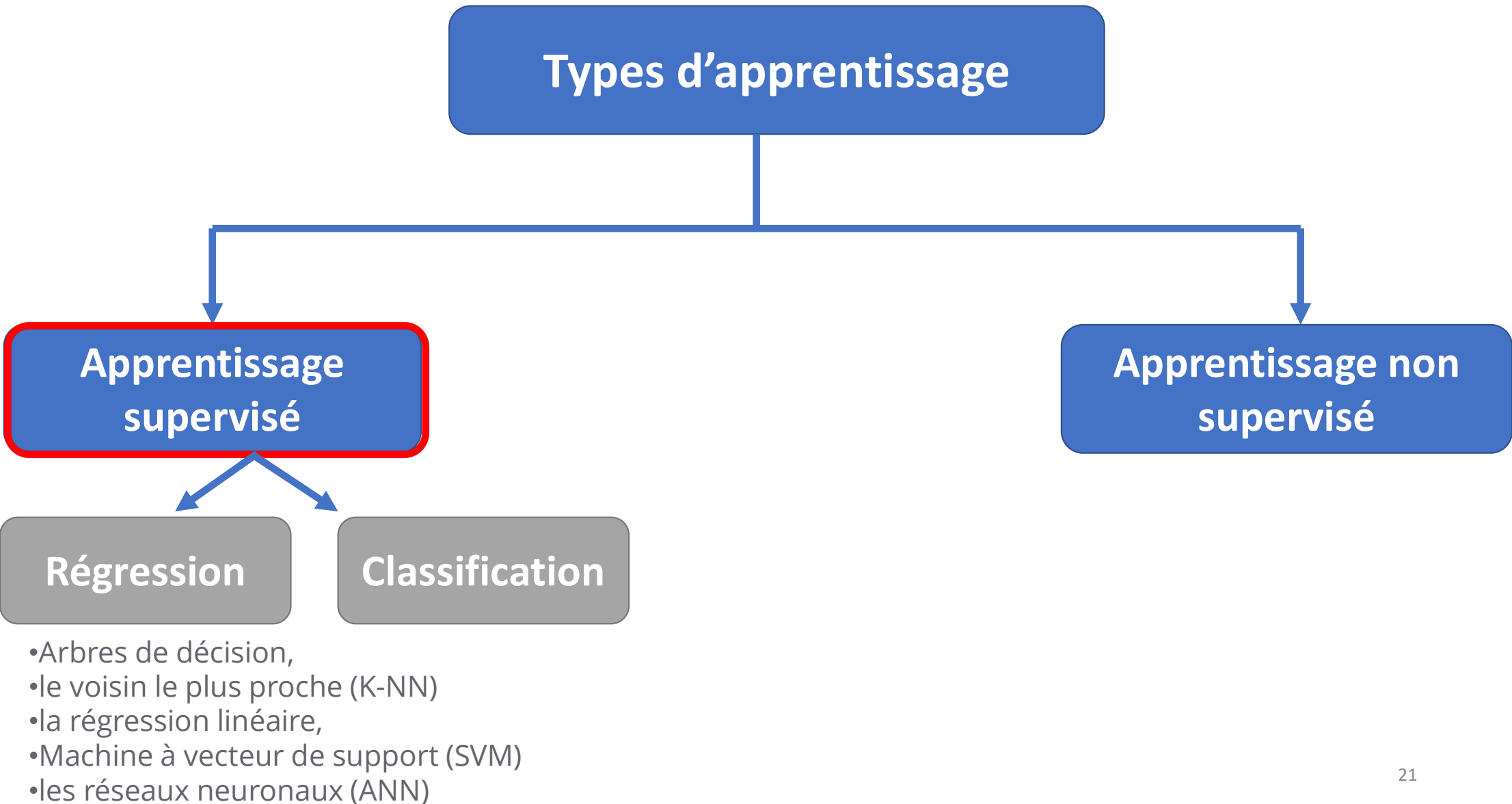
L'apprentissage supervisé se fait en présence d'un superviseur, tout comme l'apprentissage effectué par un petit enfant avec l'aide de son professeur. De même qu'un enfant est entraîné à reconnaître des fruits, des couleurs, des chiffres sous la supervision d'un enseignant, cette méthode est un apprentissage supervisé.

Dans cette méthode, chaque étape de l'enfant est contrôlée par l'enseignant et l'enfant apprend à partir du résultat qu'il doit produire.

Apprentissage supervisé

- Dans l'algorithme **ML supervisé**, le résultat est déjà connu. Il existe une correspondance entre l'entrée et la sortie.
- Par conséquent, pour créer un modèle, la machine est alimentée par de nombreuses données d'entrée d'apprentissage (dont l'entrée et la sortie correspondante sont connues).
- Les données d'apprentissage permettent d'atteindre un certain niveau de précision pour le modèle de données créé. Le modèle construit est maintenant prêt à recevoir de nouvelles données d'entrée et à prédire les résultats.

Types d'apprentissage



```
graph TD; A[Types d'apprentissage] --> B[Apprentissage supervisé]; A --> C[Apprentissage non supervisé]; B --> D[Régression]; B --> E[Classification];
```

Apprentissage supervisé

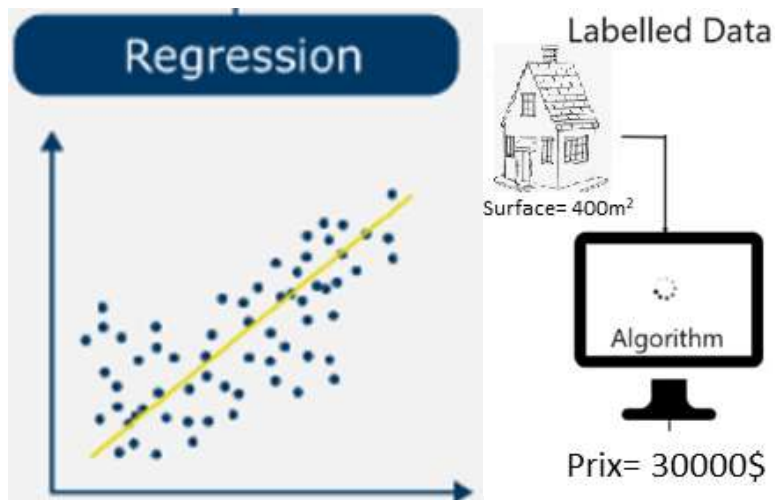
Régression

Classification

- Arbres de décision,
- le voisin le plus proche (K-NN)
- la régression linéaire,
- Machine à vecteur de support (SVM)
- les réseaux neuronaux (ANN)

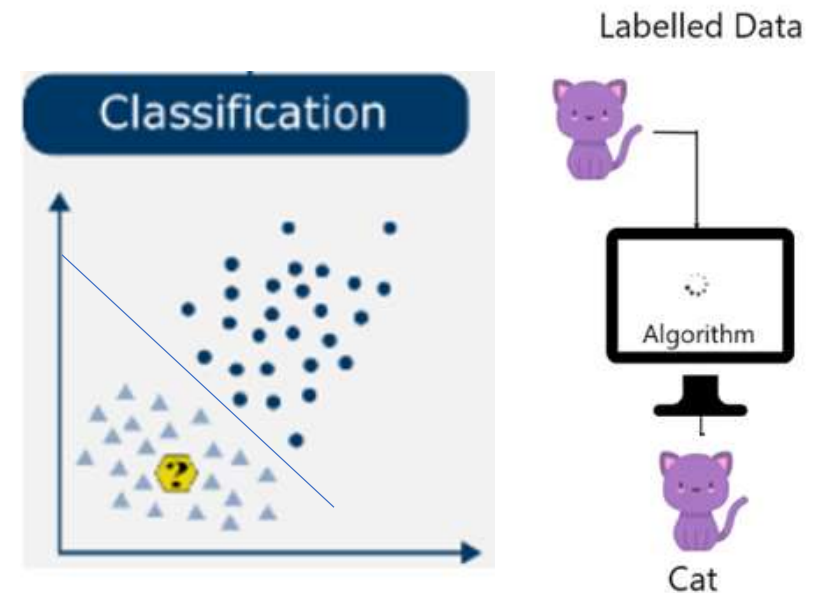
Apprentissage non supervisé

Apprentissage supervisé



Dans les problèmes de régression on cherche à prédire une valeur d'une variable continue qui prend une infinité des valeurs, par exemple le prix d'une maison

On prédire le prix d'une maison à partir de sa surface ou l'emplacement ou le nombre des chambres ...



Pour les problèmes de classification on cherche à prédire une valeur d'une variable discrète, par exemple de prédire un animal chat ou chien, chat qui va prendre la valeur 0 et chien la valeur 1.

Types d'apprentissage

```
graph TD; A[Types d'apprentissage] --> B[Apprentissage supervisé]; A --> C[Apprentissage non supervisé];
```

Apprentissage supervisé

- Arbres de décision,
- le voisin le plus proche (K-NN)
- la régression linéaire,
- Machine à vecteur de support (SVM)
- les réseaux neuronaux (ANN)

Apprentissage non supervisé

- Apriori
- le clustering K-means
- ACP

Apprentissage non supervisé

- L'apprentissage non supervisé se fait sans l'aide d'un superviseur, tout comme un poisson apprend à nager tout seul. Il s'agit d'un processus d'apprentissage indépendant.
- Dans ce modèle, comme il n'y a pas de sortie associée à l'entrée, les valeurs cibles sont inconnues/non étiquetées. Le système doit apprendre par lui-même à partir des données qu'il reçoit en entrée et détecter les modèles cachés.



Types d'apprentissage

Apprentissage
supervisé

Apprentissage non
supervisé

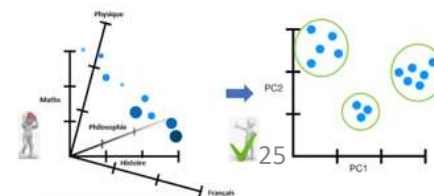
Clustering



Association



Réduction de
dimension



Apprentissage supervisé

➤ Régression

➤ Classification

Régression

Régression

- **Régression:** Un problème de régression se pose lorsque la variable de sortie est une valeur réelle, telle que «dollars» ou «poids».
- *Exemples :*
 - Prédire le prix de l'immobilier

Régression

- Pour résoudre un problème **d'apprentissage supervisé**, il faut procéder en **4 étapes** :
 1. Disposer d'un **Dataset** (**x,y**) où **x** sont les *features* et **y** est la *target*
 2. Développer un **Modèle** dont les **paramètres** sont à trouver
 3. Définir une **Fonction Coût** qui mesure l'erreur entre le modèle et les valeurs de du Dataset
 4. Utiliser un **Algorithme d'Apprentissage** qui cherche le modèle (en réalité les paramètres) qui minimisent la Fonction Coût, c'est-à-dire qui nous donne le modèle aux plus petites erreurs.

Régression

1.Dataset

2.Modèle

3.Fonction Coût

4.Algorithme d'Apprentissage

Régression

1. Dataset

- Disposer d'un **Dataset** (x, y) où x sont les *features* et y est la *target*

Exemple de Dataset sur des appartements

Target y

Features

x_1 x_2 x_3

Prix	Surface	Qualité	Adresse postale
313,000	90	3	95000
720,000	110	5	93000
250,000	40	4	44500
290,000	60	3	67000
190,000	50	3	59300
...	...	...	...

Par convention:
 m : nombre d'exemples
 n : nombre de features

Par convention, on note:
 $x_{feature}^{(exemple)}$

$x_3^{(2)}$

Régression

1.Dataset

2.Modèle

3.Fonction Coût

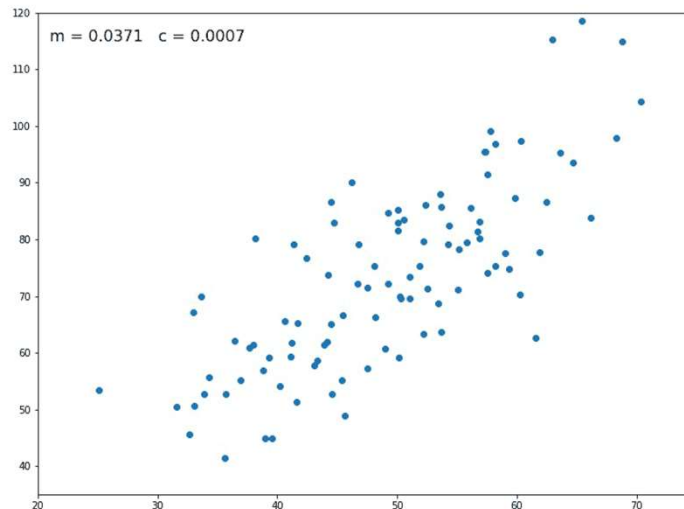
4.Algorithme d'Apprentissage

Régression

2. Le Modèle de Régression

1- Développer un **Modèle** dont les **paramètres** sont à trouver

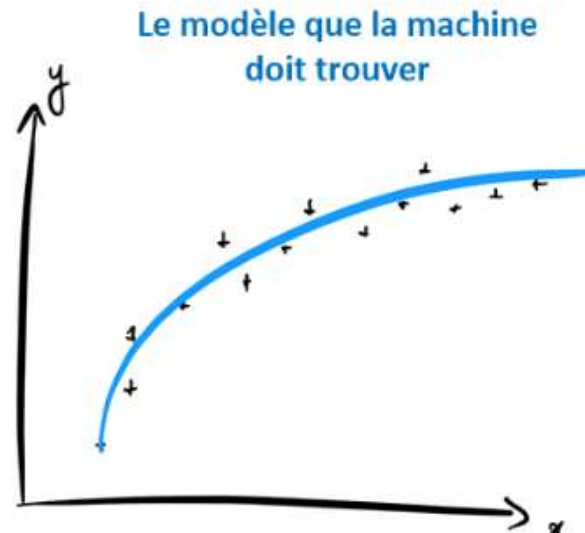
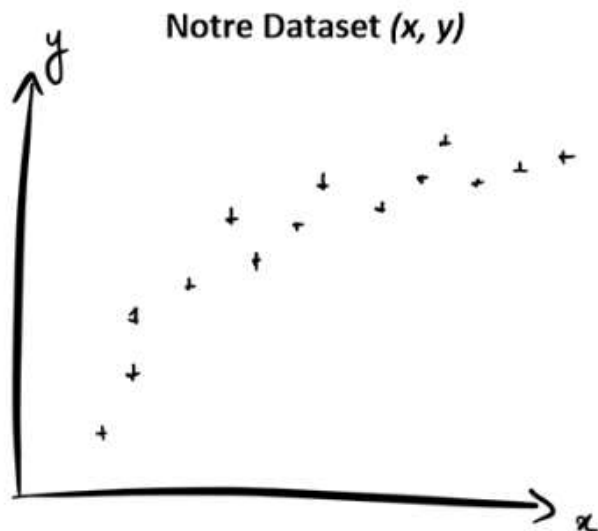
En voyant notre nuage de point, on dirait clairement qu'il suit une **tendance linéaire**, voilà pourquoi nous allons développer un modèle... **linéaire** : $f(x) = ax + b$



Régression

2. Le Modèle de Régression

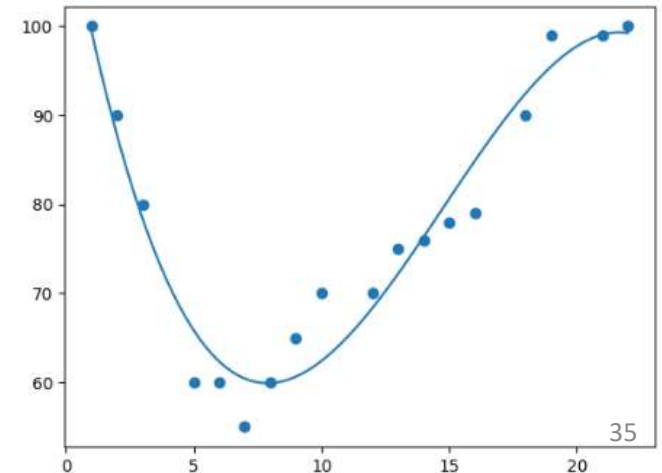
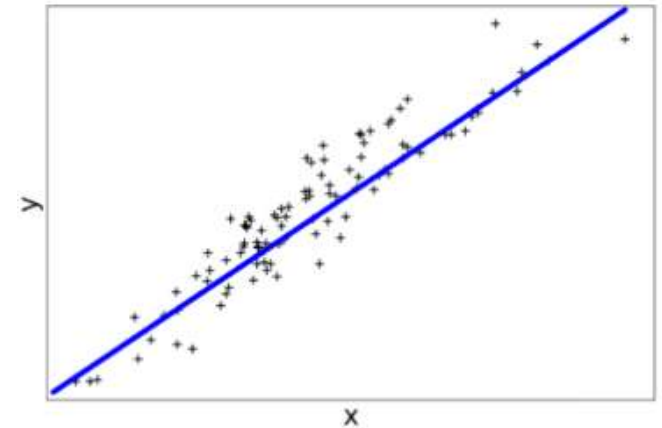
- Si par exemple votre **Dataset** vous donne le nuage de point suivant, alors la machine devra trouver le modèle qui rentre le mieux dans ce nuage de point.
- c'est à nous de choisir le type de modèle (c'est-à-dire la fonction mathématique linéaire ou linéaire) et c'est à la machine de trouver les **coefficients** de cette fonction qui donnent les meilleurs résultats. Par convention on appelle ces coefficients les **paramètres** du modèle.



Régression

2. Le Modèle de Régression

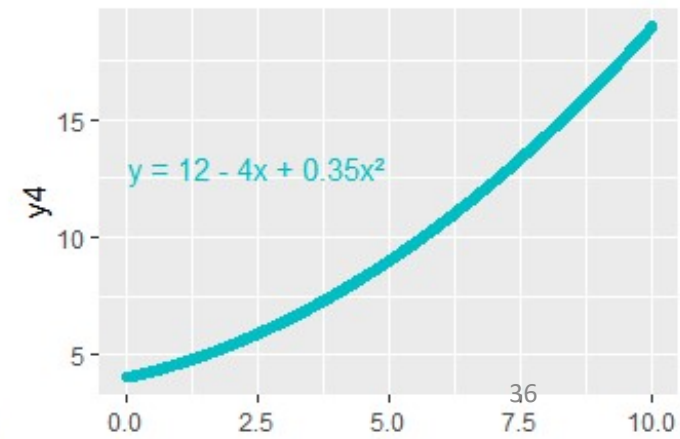
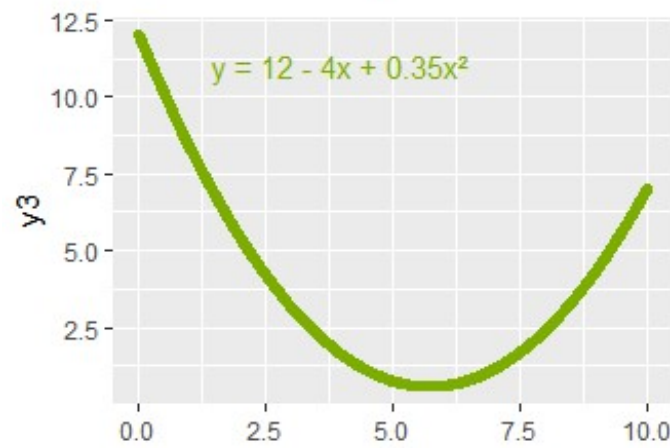
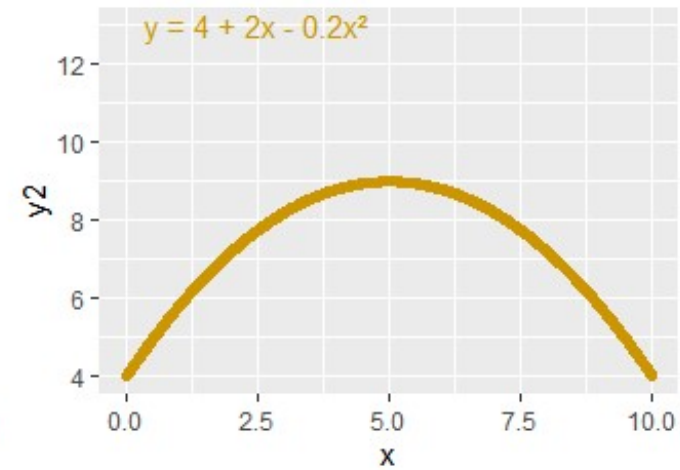
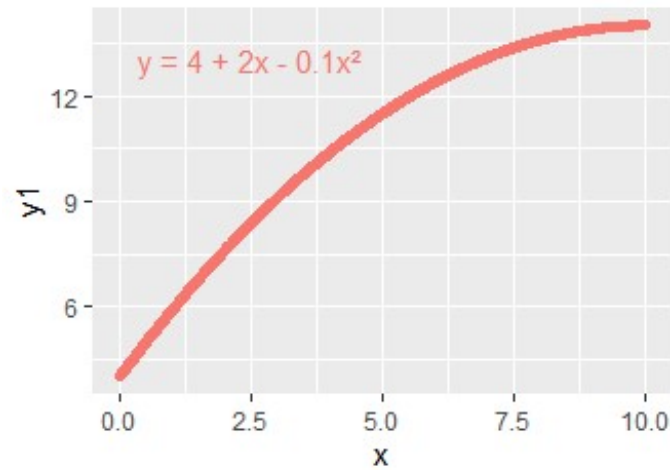
- Par exemple, on peut choisir de développer un **modèle linéaire** $f(x) = ax + b$ et on laisse la machine trouver la valeur de a et b qui donne les meilleurs résultats.
- Ou bien on peut choisir un **modèle non-linéaire** $f(x) = ax^2 + bx + c$, par exemple, où a , b et c sont les paramètres.



Régression

2. Le Modèle de Régression

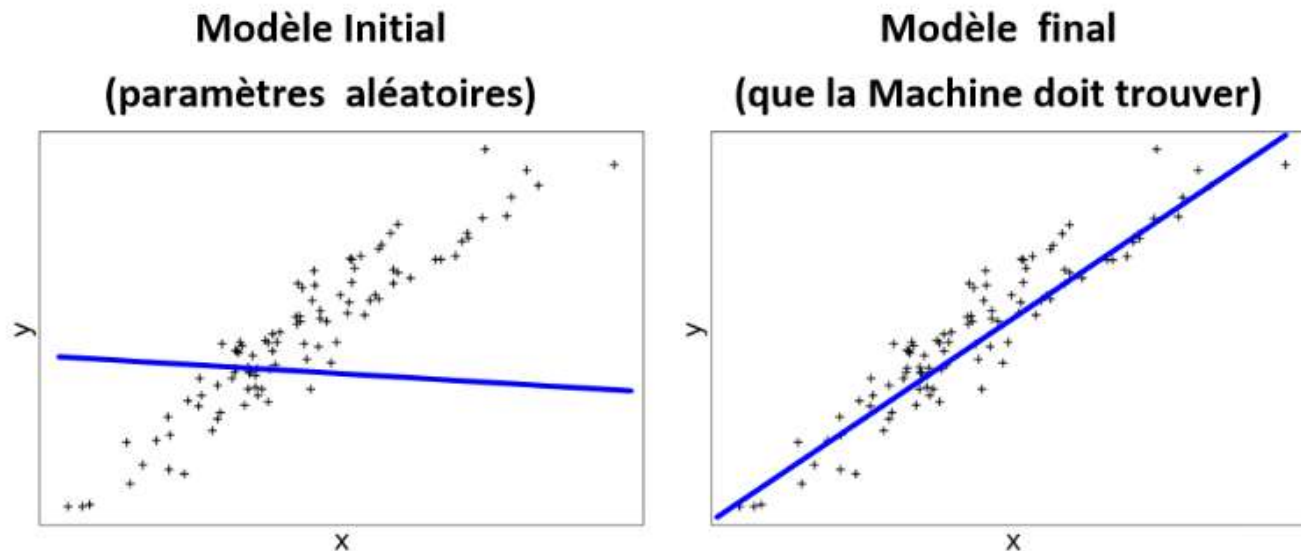
Exemple des modèles non-linéaires



Régression

2. Modèle de régression

- Nous ne connaissons pas la valeur des **paramètres** a et b , il est donc impossible de tracer une bonne droite sur le nuage de point, à moins de choisir des paramètres au hasard. Ce sera le rôle de la machine **d'apprendre** ces valeurs en minimisant la fonction coût.



Régression

1.Dataset

2.Modèle

3.Fonction Coût

4.Algorithme d'Apprentissage

Régression

3. Fonction Coût

- L'objectif du **machine learning** est de trouver un **modèle** qui effectue une **approximation de la réalité**, à l'aide de laquelle on va pouvoir effectuer des **prédictions**
- En apprentissage supervisé, la notion principale est celle de **perte d'information** (*loss* en anglais) due à l'approximation du modèle par rapport aux données.
- L'apprentissage se résume en fait souvent à une méthode itérative qui converge vers un **minimum** de cette fonction. La distance la plus utilisée pour mesurer l'éloignement entre le modèle et les données est **l'erreur quadratique** (la distance euclidienne entre un point et le modèle).

Régression

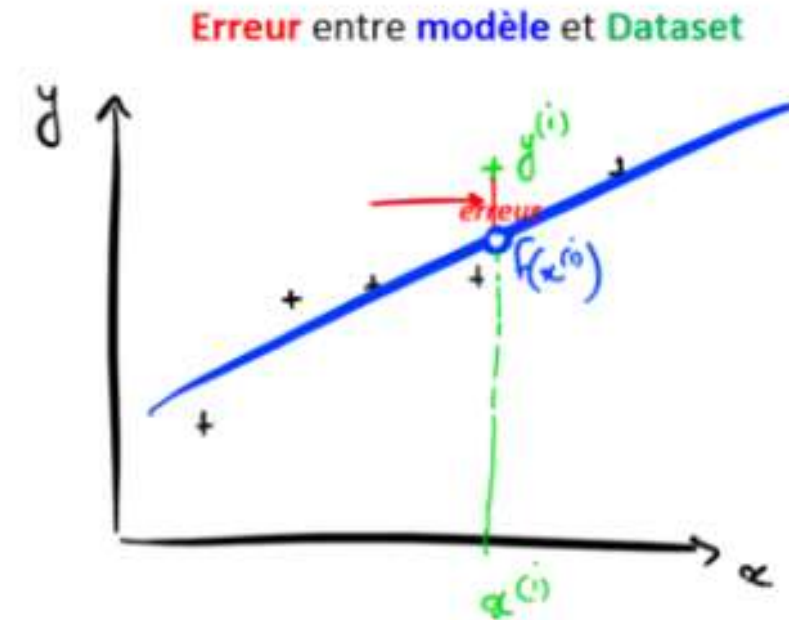
3. Fonction Coût

- Comment faire pour que la machine trouve le meilleur modèle ?
- Autrement dit, comment faire pour que la machine apprenne ?
- Pour que la machine trouve le meilleur modèle, il faut déjà qu'elle puisse **mesurer la performance** d'un modèle donné.
- Pour savoir quel modèle est le meilleur parmi 2 candidats, il faut les **évaluer**. Pour cela, on mesure l'**erreur** entre un modèle et le **Dataset**, et on appelle ça la **Fonction Coût**.

Régression

3. Fonction Coût

- Dans le cas d'une **régression**, on peut par exemple mesurer l'**erreur** entre la **prédiction du modèle** $f(x^{(i)})$ et la valeur $y^{(i)}$ qui est associée à ce $x^{(i)}$ dans notre **Dataset**.
- **Erreur** = $f(x^{(i)}) - y^{(i)}$



Régression

3. Fonction cout: Erreur Quadratique Moyenne

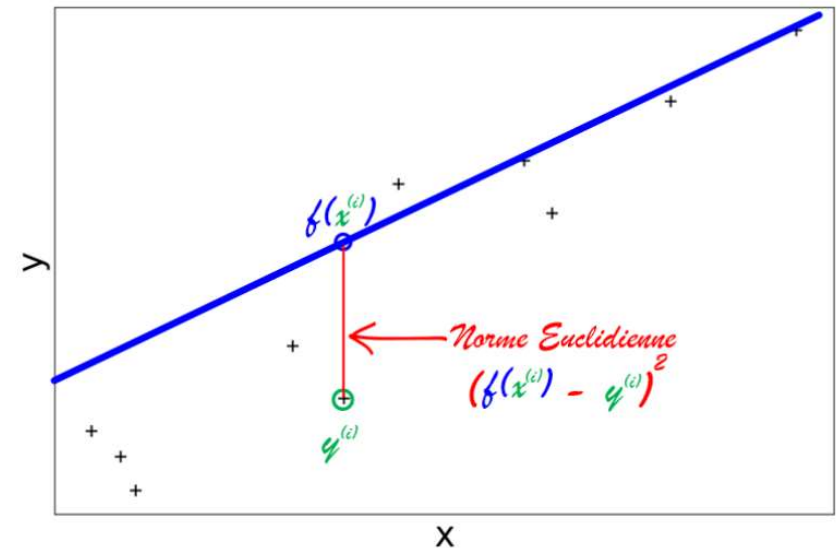
- La fonction coût nous permet d'évaluer la performance de notre modèle en mesurant les erreurs entre $f(x)$ et y . La question que l'on se pose est: Comment mesurer ces erreurs?
- Imaginez que vous avez un objet de taille **1,4m**. Votre modèle de Machine Learning prédit que cet objet de taille **0,3m**. Vous pourriez conclure que votre modèle fait donc une **erreur** de **1,4-0,3= 1,1m**.
- Ainsi, on pourrait se dire que pour mesurer nos erreurs, il faut faire calculer la différence $f(x) - y$. Cependant, si votre prédiction $f(x)$ est inférieure à y , alors cette erreur est négative, ce qui n'est pas pratique (personne ne dit : » *J'ai fait une erreur de négatif*)

Régression

3. Fonction cout: Erreur Quadratique Moyenne

- Pour mesurer les **erreurs** entre les **prédictions** $f(x)$ et les **valeurs** y du **Dataset**, on calcule le carré de la différence : $(f(x) - y)^2$. C'est au passage ce qu'on appelle la **norme euclidienne**, qui représente la distance directe entre $f(x)$ et y dans la géométrie euclidienne (la géométrie qu'on utilise au quotidien)
- Pour la régression linéaire, la **fonction Coût** va être la **moyenne** de toutes nos erreurs, c'est-à-dire :

$$J = \frac{[f(x^{(1)}) - y^{(1)}]^2 + \dots + [f(x^{(m)}) - y^{(m)}]^2}{m}$$



Régression

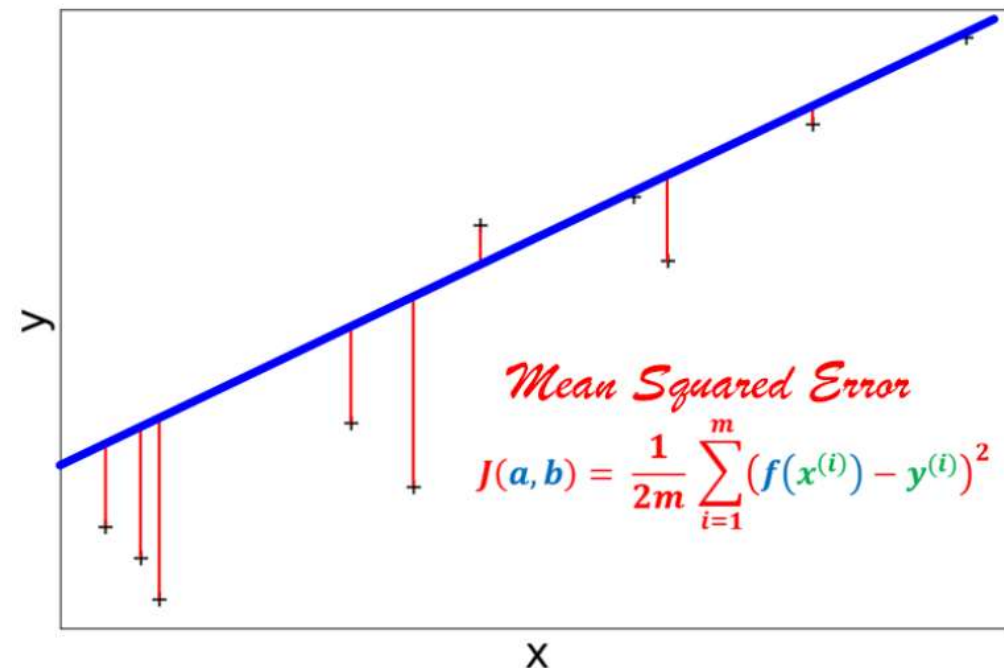
3. Fonction cout: Erreur Quadratique Moyenne

- Par convention on écrit cette fonction de la manière suivante, en rajoutant un coefficient **1/2** pour simplifier un calcul de dérivée qui viendra par la suite.

$$J(a, b) = \frac{1}{2m} \sum_{i=1}^m [f(x^{(i)}) - y^{(i)}]^2$$

$$J(a, b) = \frac{1}{2m} \sum_{i=1}^m [ax^{(i)} + b - y^{(i)}]^2$$

C'est **l'Erreur Quadratique Moyenne (EQM)**



Régression

3. Fonction cout: Erreur Quadratique Moyenne

- Par exemple si vos valeurs sont les suivantes :

$$y = [1, 1.5, 1.2, 0.9, 1]$$

- Et que les valeurs prédites par votre modèle sont les suivantes :

$$y_{\text{pred}} = [1.1, 1.2, 1.2, 1.3, 1.2]$$

Calculer l'erreur quadratique moyenne?

Exemple

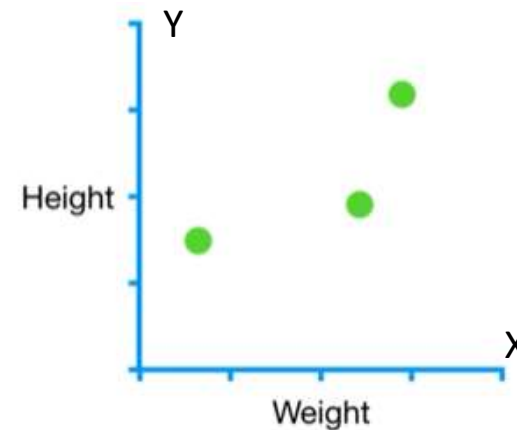
Régression

DataSet

Nous allons voir comment développer un **modèle** de Machine Learning à partir d'un **DataSet** à une seule variable x . On aura donc un **DataSet** avec **m=3** exemples et **1** « *feature* » variable.

Notre **DataSet** représente la taille en fonction du poids. Ce **DataSet** nous pourrait nous donner le nuage de point suivant :

Weight (x)	Height (y)
0,5	1,4
2,3	1,9
2,9	3,2

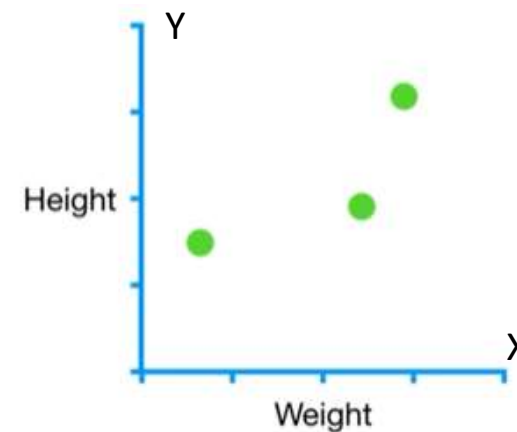


Régression

Modèle

En voyant notre nuage de point, on dirait clairement qu'il suit une **tendance linéaire**, voilà pourquoi nous allons développer un modèle... **linéaire** ! Notre modèle $f(x) = ax + b$

Weight (x)	Height (y)
0,5	1,4
2,3	1,9
2,9	3,2



Régression

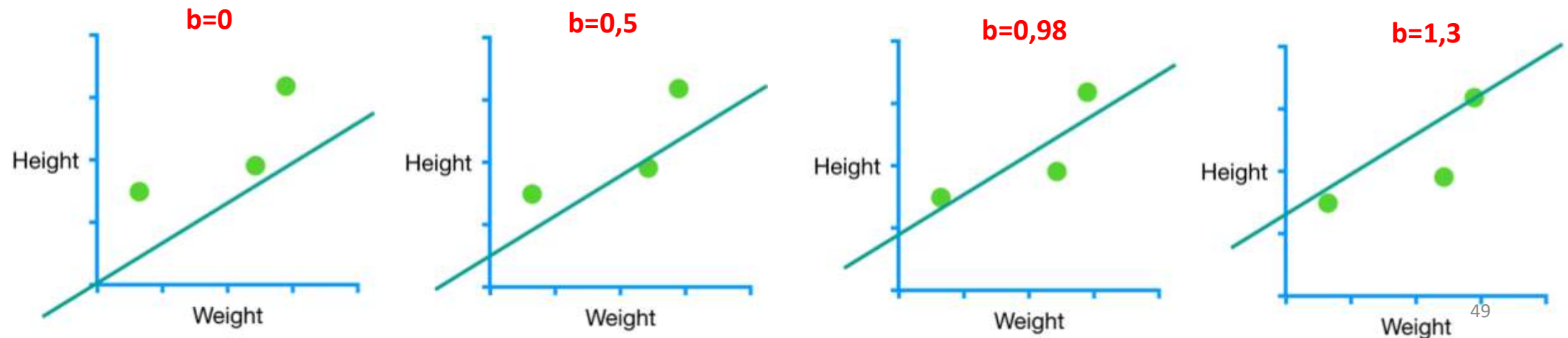
Fonction cout

objet	Weight (x)	Height (y)
Obj1	0,5	1,4
Obj2	2,3	1,9
Obj3	2,9	3,2

$$y = ax + b$$

Nous allons voir comment le ML peut ajuster la ligne (le modèle) aux données en trouvant les valeurs optimales des paramètres a et b du modèle
Tout d'abord nous commençons par fixer la valeur de a et nous essayons de voir comment le ML va trouver la valeur optimale de b

Pour commencer on fixe par exemple $a=0,64$ et on change la valeur de b



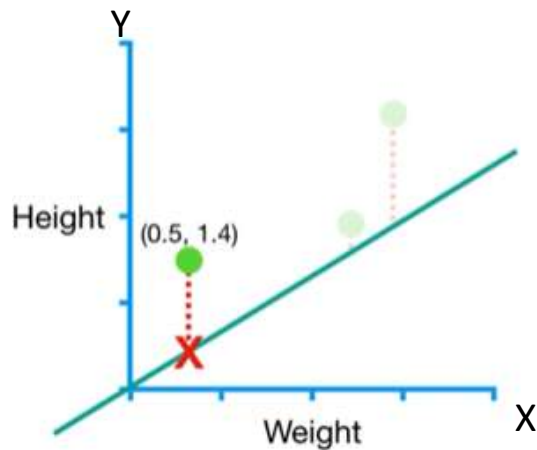
Régression

Fonction cout

$a=0,64$ et $b=0$, calculons l'erreur

objet	Weight (x)	Height (y)
Obj1	0,5	1,4
Obj2	2,3	1,9
Obj3	2,9	3,2

- Le **premier point** de données représente l'objet 1 avec un poids **0,5** et une taille **1,4**
- Sur la ligne de prédiction (le modèle), on a pour un objet de poids $x=0,5$ la taille prédite $f(x) = 0,64*x + 0 = 0,64*0,5 + 0 = 0,32$
- L'erreur= (taille observée – taille prédite)= $(1,4-0,32)=1,1$
- $EQ=1,1^2$

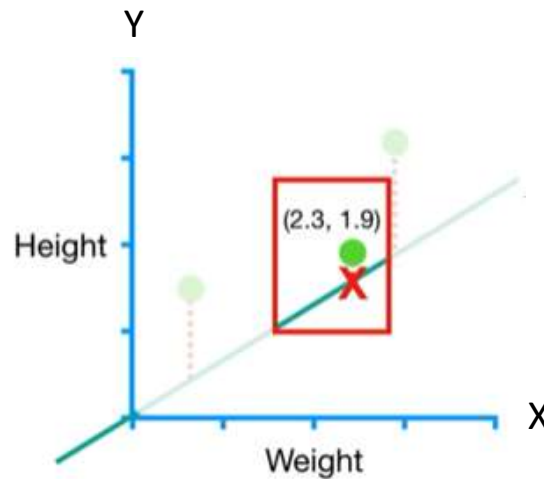
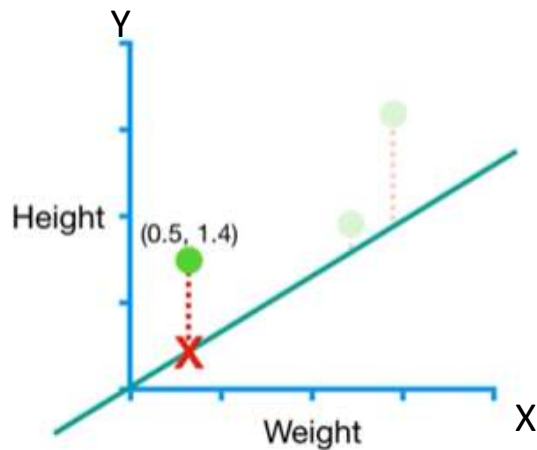


Régression

Fonction cout

objet	Weight (x)	Height (y)
Obj1	0,5	1,4
Obj2	2,3	1,9
Obj3	2,9	3,2

- Le **deuxième point** de données représente l'objet 2 avec un poids **2,3** et une taille **1,9**
- Sur la ligne de prédiction (le modèle) on a pour un objet de poids $x=2,3$ la taille prédite $f(x) = 0,64 \cdot x + 0 = 0,64 \cdot 2,3 + 0 = 1,5$
- **L'erreur = (taille observée – taille prédite) = (1,9 - 1,5) = 0,4**
- **EQ = $1,1^2 + 0,4^2$**

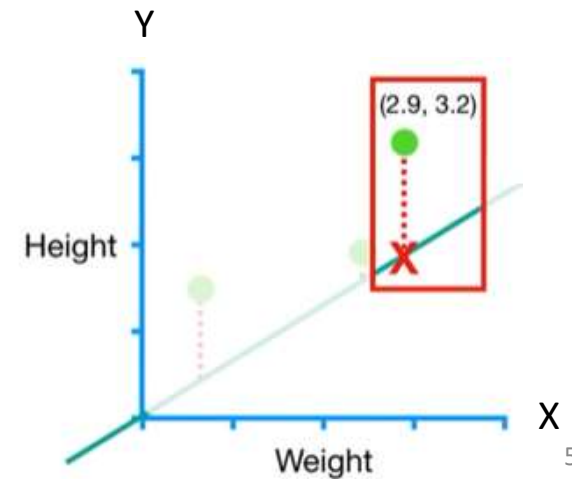
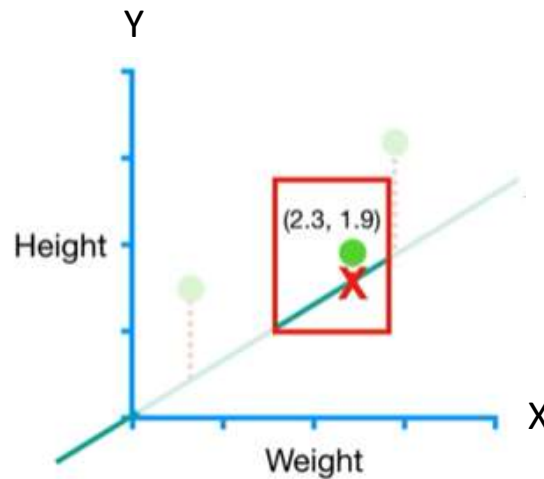
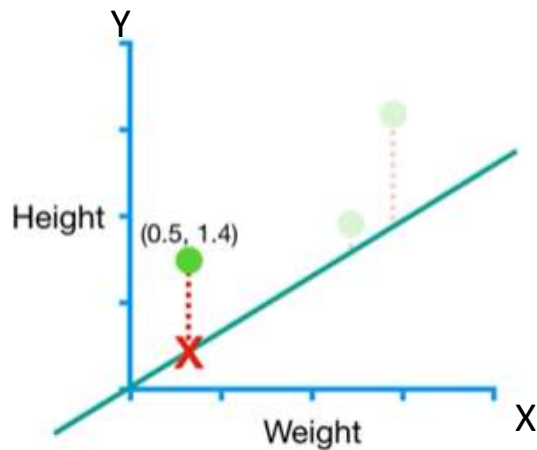


Régression

Fonction cout

objet	Weight (x)	Height (y)
Obj1	0,5	1,4
Obj2	2, 3	1,9
Obj3	2,9	3,2

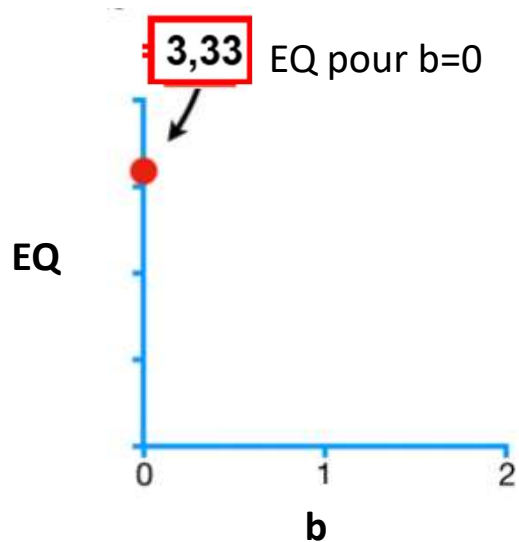
- Le troisième point de données représente l'objet 3 avec un poids **2,9** et une taille **3,2**
- Sur la ligne de prédiction (le modèle) on a pour un objet de poids $x=2,9$ la taille prédite $f(x) = 0,64 \cdot x + 0 = 0,64 \cdot 2,9 + 0 = 1,8$
- **L'erreur = (taille observée – taille prédite) = (3,2 - 1,8) = 1,4**
- **EQ = $1,1^2 + 0,4^2 + 1,4^2 = 3,33$**



Régression

Fonction cout

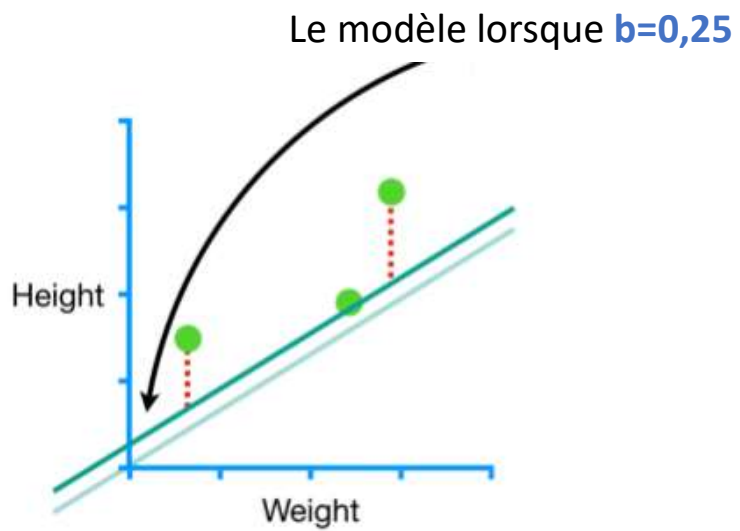
➤ $EQ = 1,1^2 + 0,4^2 + 1,4^2 = 3,33$



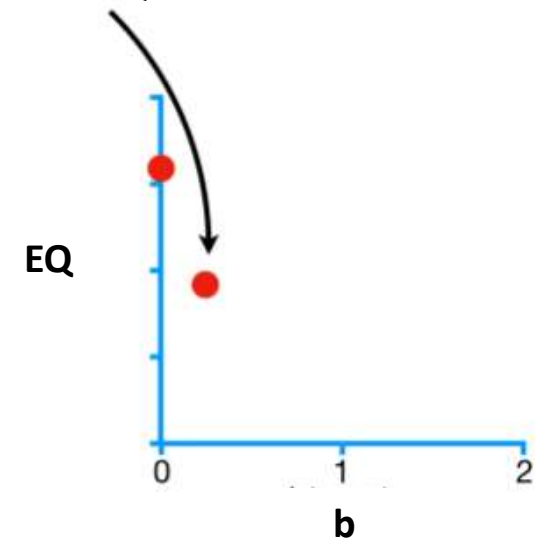
Le graphe représente la somme d'Erreur Quadratique en fonction de b

Régression

Fonction cout

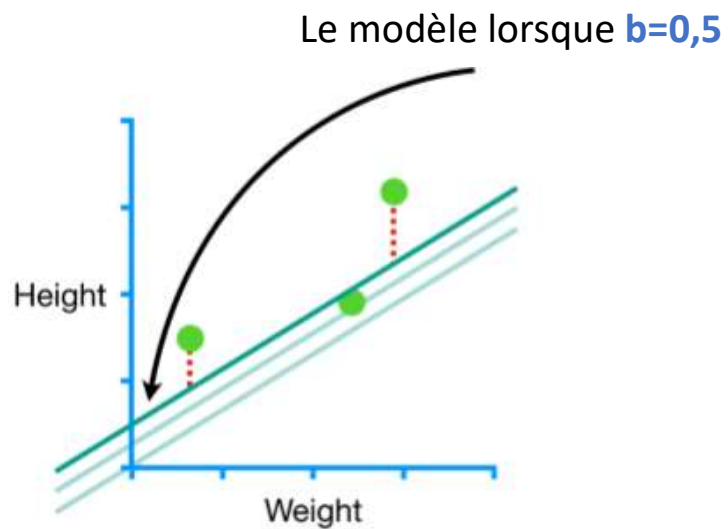


Nous obtenons EQ ici

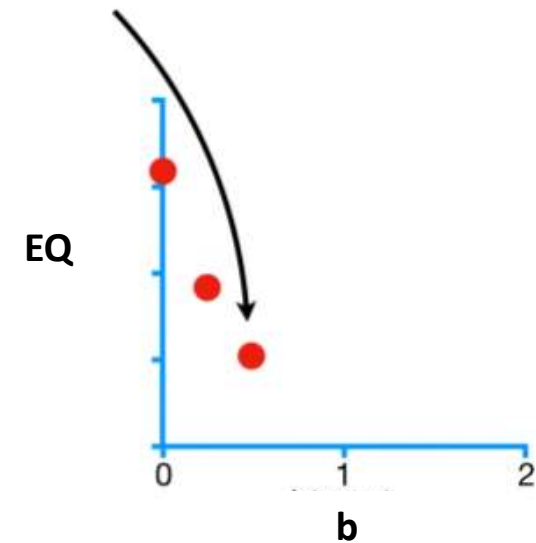


Régression

Fonction cout



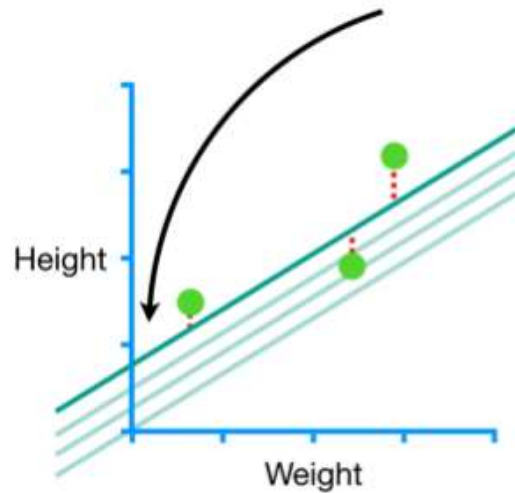
Nous obtenons EQ ici



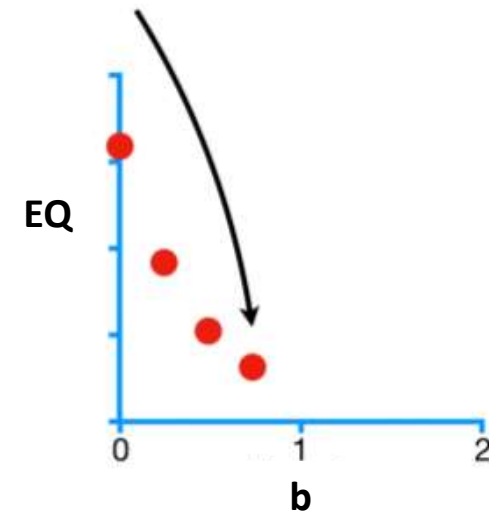
Régression

Fonction cout

En augment la valeur de b



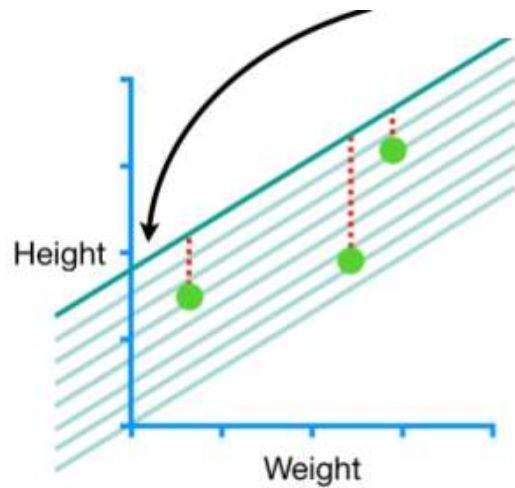
Nous obtenons Les EQ suivantes



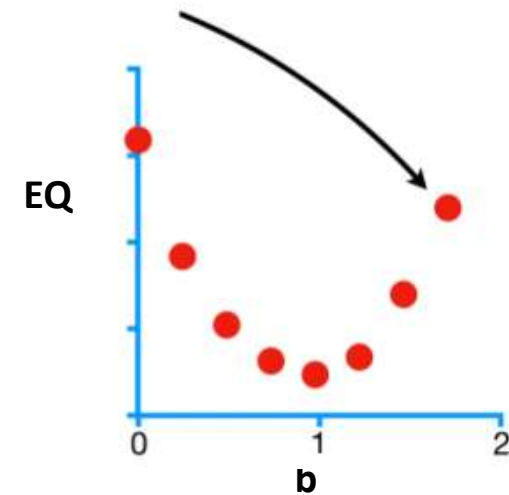
Régression

Fonction cout

En augmentant encore les valeurs de b



Nous obtenons Les EQ suivantes



Régression

1.Dataset

2.Modèle

3.Fonction Coût

4.Algorithme d'Apprentissage

Régression

4. L'Algorithme d'apprentissage

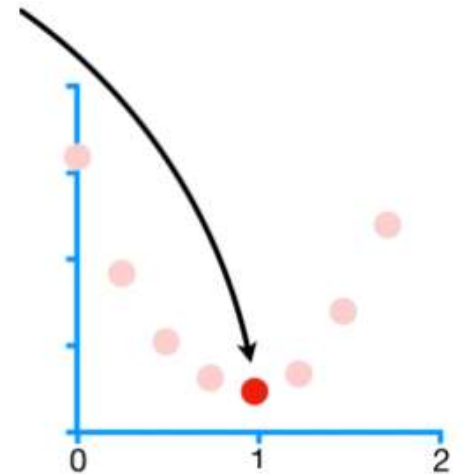
En Supervised Learning, la machine cherche les **paramètres** (a et b) de modèle $ax^{(i)} + b$ qui **minimisent** la **Fonction Coût**

$$J(a, b) = \frac{1}{2m} \sum_{i=1}^m [ax^{(i)} + b - y^{(i)}]^2.$$

C'est ça qu'on appelle **l'apprentissage**.

Cette phrase est **très importante**. C'est l'essentiel de ce qu'il faut comprendre en Machine Learning.

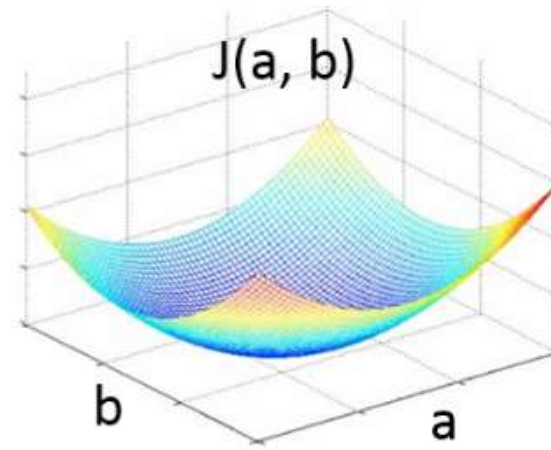
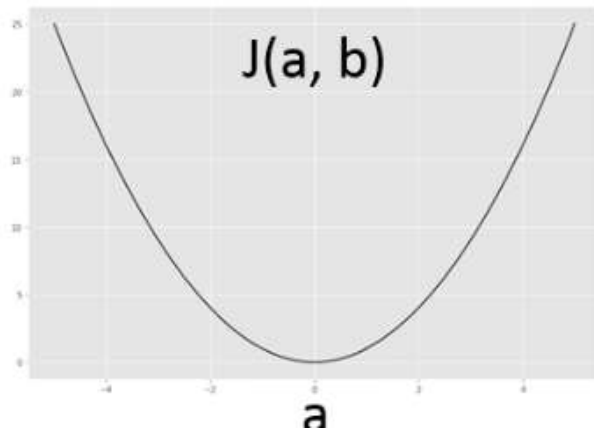
EQM minimale



Régression

4. L'Algorithme d'apprentissage

- La **Fonction Coût** $J(a, b) = \frac{1}{2m} \sum_{i=1}^m [ax^{(i)} + b - y^{(i)}]^2$ ne dépend que de deux paramètres : **a** et **b**.
- Quand on l'observe, elle a l'apparence d'une vallée bien lisse. On dit que c'est une fonction **convexe**. Cette propriété est très importante pour s'assurer de converger vers le minimum avec l'algorithme de la **descente de gradient**



Régression

4. L'Algorithme d'apprentissage

- On dit que la machine apprend quand elle trouve quels sont les paramètres du modèle qui **minimisent** la fonction Coût.
- On cherche donc à développer un **algorithme de minimisation**. Il existe un paquet de méthodes de minimisation (méthode des **moindres carrés**, méthode de **Newton**, **Gradient Descent**, **Simplex**, etc.)
- Pour un modèle de régression linéaire, on utilise le plus souvent la **méthode des moindres carrés** (quand le problème est simple) et l'algorithme de **Gradient Descent** (pour les régressions plus compliquée).
- Le **Gradient Descent**, que nous allons décortiquer en détail dans les prochains slides, permet de converger progressivement vers le minimum de n'importe quelle fonction **convexe** (comme la Fonction Coût) en suivant la direction de la pente (le gradient) qui descend, d'où son nom de Gradient Descent.

Régression

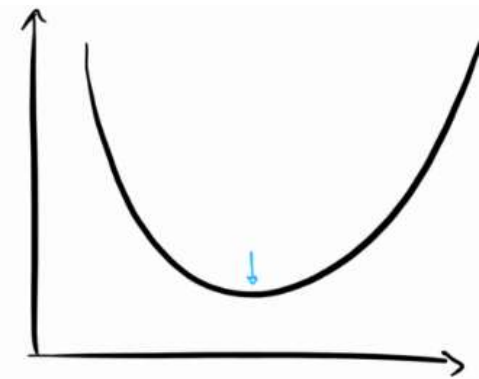
L'Algorithme d'apprentissage: Descente de gradient

- La **Descente de Gradient**, (ou *Gradient Descent* en anglais) est un des algorithmes **les plus importants** de tout le Machine Learning et de tout le Deep Learning.
- Il s'agit d'un algorithme **d'optimisation** extrêmement puissant qui permet d'entraîner les modèles de **régression linéaire, régression logistiques** ou encore les **réseaux de neurones**.
- Si vous vous lancez dans le Machine Learning, il est donc **impératif** de comprendre en profondeur l'algorithme de la descente de gradient

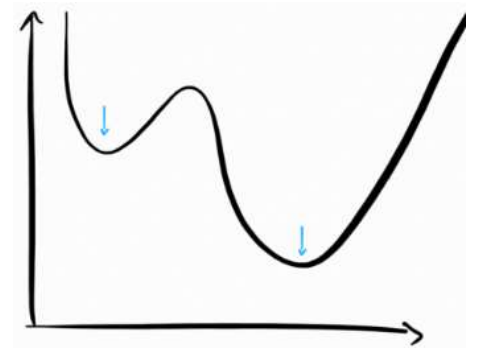
Régression

L'Algorithme d'apprentissage: Descente de gradient

- La **Descente de Gradient** est un algorithme **d'optimisation** qui permet de trouver le **minimum** de n'importe quelle fonction **convexe** en convergeant progressivement vers celui-ci.
- *Rappel: Une fonction **convexe** est une fonction dont l'allure ressemble à celle d'une belle vallée avec au centre un **minimum global**. A l'inverse, une fonction **non-convexe** est une fonction qui présente plusieurs **minimaux locaux** et l'algorithme de descente de gradient ne doit pas être utilisé sur ces fonctions, au risque de se bloquer au premier minima rencontré.*



Fonction Convexe

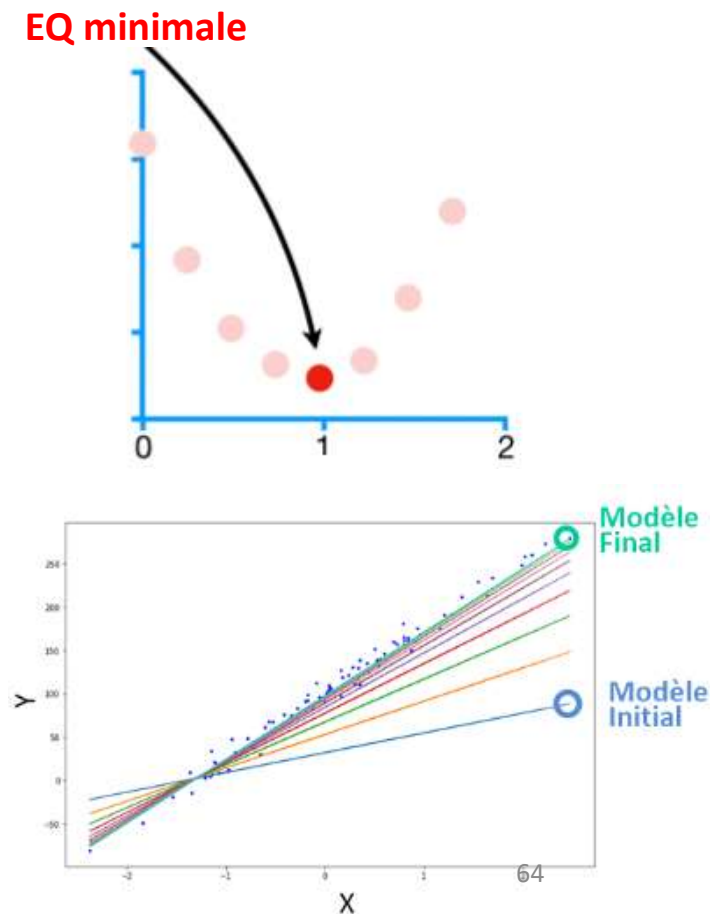


Fonction Non-Convexe

Régression

L'Algorithme d'apprentissage: Descente de gradient

- En Machine Learning, on va utiliser l'algorithme de la Descente de Gradient dans les problèmes **d'apprentissage supervisé** pour **minimiser la fonction coût**, qui justement est une fonction **convexe** (par exemple l'*erreur quadratique moyenne*).
- C'est grâce à cet algorithme que la machine apprend, c'est-à-dire trouve le meilleur modèle. En effet, rappelez-vous que minimiser la fonction coût revient à trouver les paramètres **a, b, c, etc.** qui donnent les plus petites erreurs entre notre modèle et les points du Dataset.



Régression

L'Algorithme d'apprentissage: Descente de gradient

□ Trouver le minimum

Étape 1 : Calcul de la dérivée de la Fonction Coût

- Nous partons d'un point initial aléatoire puis nous mesurons la valeur de la pente en ce point. Et comment mesure-t-on une pente en mathématique ? En calculant la **dérivée** de la fonction !

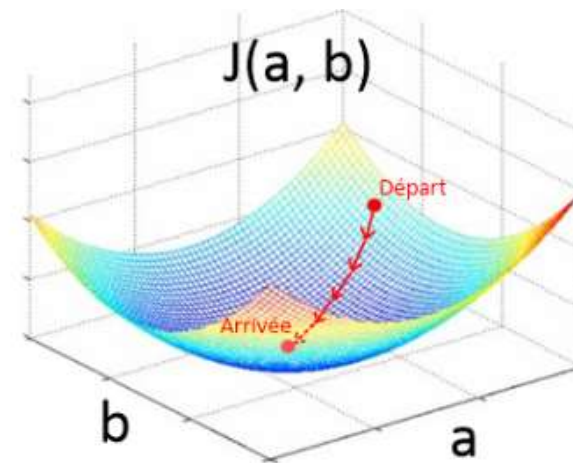
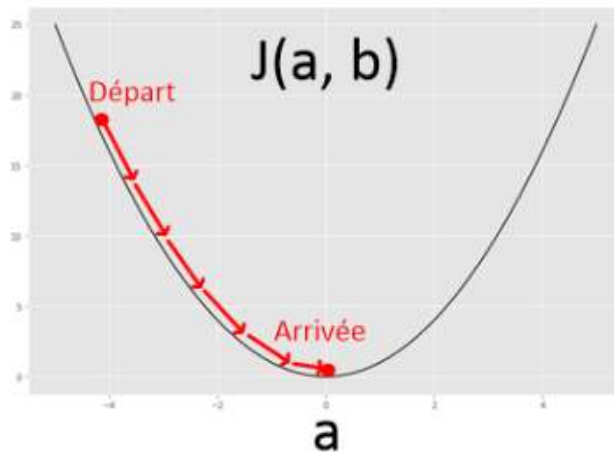
Étape 2 : Mise à jour des paramètres du modèle

- On progresse ensuite d'une certaine distance dans la direction de la pente qui descend, On appelle cette distance **Learning Rate**, que l'on pourrait traduire par **vitesse d'apprentissage**.
- Cette opération a pour résultat de modifier la valeur des paramètres (**a** et **b**) de notre modèle

Régression

L'Algorithme d'apprentissage: Descente de gradient

- En répétant ces deux étapes en boucle, l'algorithme de **Gradient Descent** est donc un algorithme **itératif**. Pour l'illustrer sur un graphique, je vais prendre l'exemple de la fonction coût **$J(a, b)$** que l'on a développé pour une régression linéaire. L'algorithme permet de trouver la valeur idéale pour les paramètres **a** et **b** .



Régression

L'Algorithme d'apprentissage: Descente de gradient

❖ Itération $i=0$

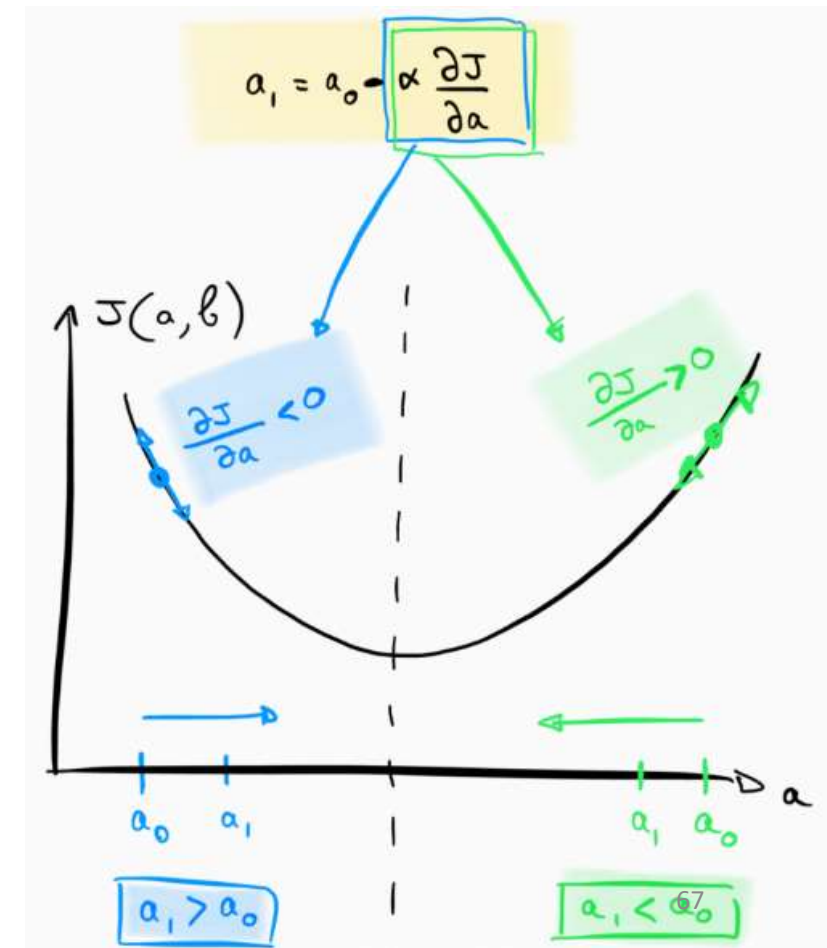
on prend une valeur aléatoire pour a_0
nous calculons la position a_1 en
appliquant la formule :

$$a_1 = a_0 - \alpha * \frac{\partial J(a, b)}{\partial a}$$

En général

• Itération suivante

- $a_{k+1} = a_k - \alpha * \frac{\partial J(a, b)}{\partial a}$



Régression

L'Algorithme d'apprentissage: Descente de gradient

- **Descente de gradient** est un algorithme itératif à chaque itération on calcule paramètres (**a** et **b**) de notre modèle
- On progresse avec une distance **α (Learning Rate)** dans la direction de la pente qui descend,

$$a_{k+1} = a_k - \alpha * \frac{\partial J(a, b)}{\partial a}$$

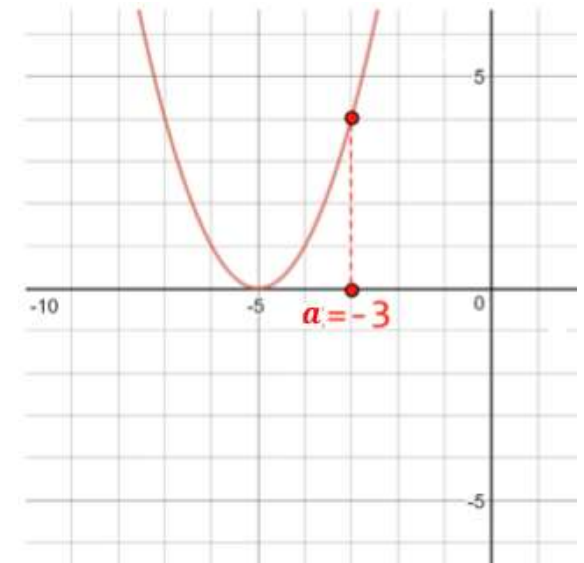
$$b_{k+1} = b_k - \alpha * \frac{\partial J(a, b)}{\partial b}$$

Régression

L'Algorithme d'apprentissage: Descente de gradient

Exemple :

- Considérons la fonction suivante $J(a) = (a + 5)^2$
- Commençons par un point aléatoire $a=-3$ et nous nous déplaçons dans la direction négative pour chercher le minimum
- Nous calculons la dérivée de la fonction
- $\frac{dJ}{da} = 2(a + 5)$



Nous déplaçons dans la direction négative un pas $\alpha = 0,01$
(`learning_rate`)

Régression

L'Algorithme d'apprentissage: Descente de gradient

Exemple :

Itération 0: calculer a_0

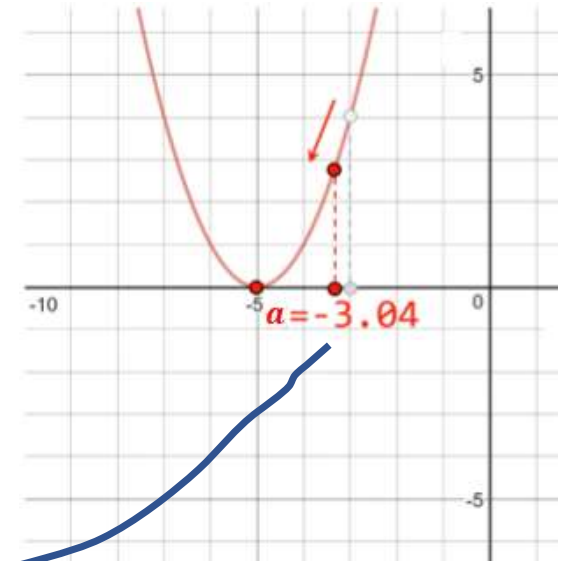
$$a_0 = -3, \frac{dJ}{da} = 2(a + 5) \text{ et learning_rate } \alpha = 0,01$$

Itération 1: calculer a_1

$$a_1 = a_0 - \alpha * \frac{dJ}{da}$$

$$a_1 = a_0 - \alpha * 2(a_0 + 5)$$

$$a_1 = -3 - 0,01 * 2(-3 + 5) = -3,04$$



Régression

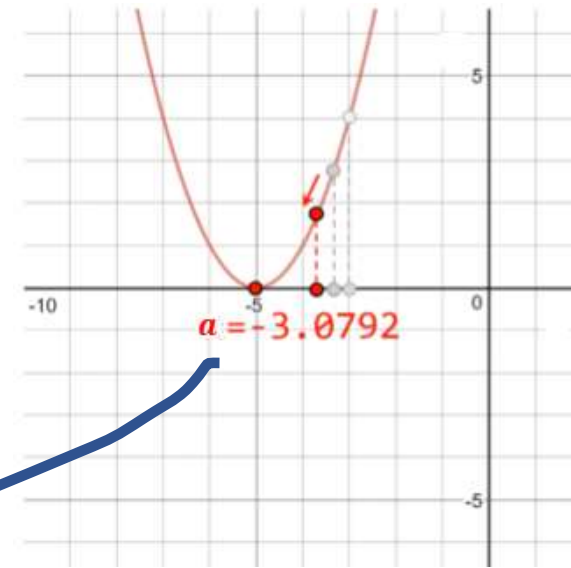
L'Algorithme d'apprentissage: Descente de gradient

Exemple :

Itération 2: calculer a_2

$$a_2 = a_1 - \alpha * 2(a_1 + 5)$$

$$a_2 = -3,04 - 0,01 * 2(-3,04 + 5) = -3,0792$$



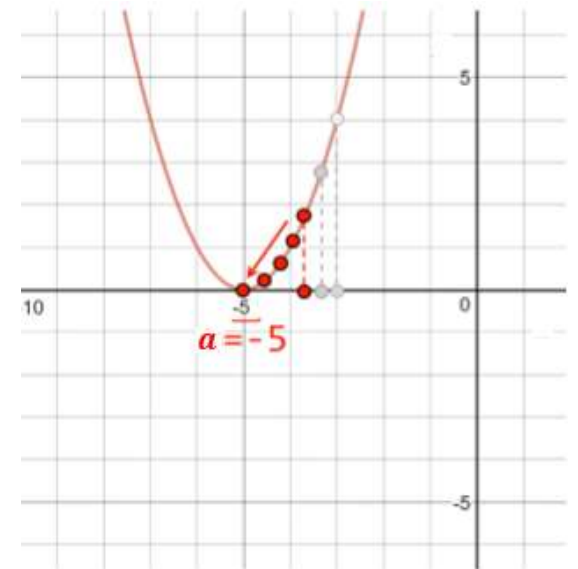
Régression

L'Algorithme d'apprentissage: Descente de gradient

Exemple :

Itération k: $a_k = -5$ nous calculons a_{k+1}

- $a_{k+1} = a_k - \alpha * 2(a_k + 5)$
- $a_{k+1} = -5 - 0,01 * 2(-5 + 5) = -5$



Implémentation de l'algorithme de régression Linéaire en Python

Importer les packages Numpy et Matplotlib.pyplot

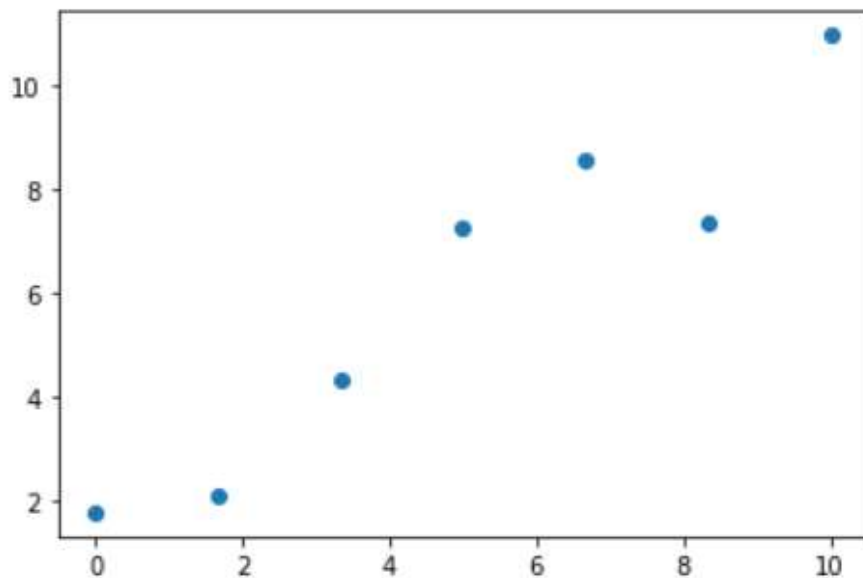
- Importer les packages **Numpy** et **Matplotlib.pyplot**.
 - **Numpy** permet de créer des **matrices** et effectuer des opérations mathématiques.
 - **Matplotlib** permet de créer des **graphiques** pour observer facilement notre dataset ainsi que le modèle construit à partir de celui-ci.

Génération d'un Dataset

- Nous allons reformuler nos équations sous **forme matricielle** pour calculer **d'une seule traite** le **Gradient**, la **Fonction Coût**, et le **modèle** sur toutes les données de notre Dataset. Voici comment procéder.
- Notre **Dataset** (x, y) contient m **exemples** et n **features**, on transfère toutes les données $y^{(i)}$ dans un vecteur Y de dimension $m \times 1$ et toutes les données $x^{(i)}$ dans une matrice X à laquelle on ajoute une colonne **biais** (c'est une colonne remplie de « 1 »). La matrice X est donc de dimension $m \times n \times 1$ et. Dans le cas de la régression linéaire à une seule variable, on a $n = 1$ et donc la matrice est de dimension $m \times 2$.

Génération d'un Dataset

```
np.random.seed(0) # pour toujours reproduire le meme dataset  
  
n_samples = 7 # nombre d'echantillons a générer  
x = np.linspace(0, 10, n_samples).reshape((n_samples, 1))  
y = x + np.random.randn(n_samples, 1)  
plt.scatter(x, y) # afficher les résultats. x en abscisse et y en ordonnée  
plt.show()
```



La fonction **linspace** de **Numpy**, nous créons un tableau de données qui présente une tendance linéaire.

La fonction **random.randn** permet d'ajouter un « bruit » **aléatoire normal** aux données. Pour effectuer un calcul matriciel correct, il est important de confier 2 dimensions (7 lignes, 1 colonne) à ces tableaux en utilisant la fonction **reshape(7, 1)**

Implémentation de l'algorithme de régression Linéaire

□ Le modèle $F = X\theta$

- le modèle linéaire $f(x^i) = ax^{(i)} + b$ ne permet que d'effectuer le calcul d'une prédiction à la fois, selon une valeur x^i unique. Pour remédier à ce problème, il suffit de créer un vecteur F de dimension $m \times 1$ qui contient toutes les prédictions :

$$F = \begin{bmatrix} f(x^{(1)}) \\ f(x^{(2)}) \\ f(x^{(3)}) \\ \dots \\ f(x^{(m)}) \end{bmatrix}$$

- Pour calculer ce vecteur, on utilise la formule suivante : $F = X\theta$ où θ est le **vecteur Paramètre** qui contient tous les paramètres de notre modèle.

Implémentation de l'algorithme de régression Linéaire

$$\underline{F} = X.\theta = \begin{bmatrix} x^{(1)} & 1 \\ \dots & \dots \\ x^{(m)} & 1 \end{bmatrix} \cdot \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} ax^{(1)} + 1 \times b \\ \dots \\ ax^{(m)} + 1 \times b \end{bmatrix} = \begin{bmatrix} f(x^{(1)}) \\ \dots \\ f(x^{(m)}) \end{bmatrix}$$

Initialiser des paramètres dans un vecteur theta.

```
# création d'un vecteur parametre theta
theta = np.random.randn(2, 1)
print(theta)

[[ -0.15135721]
 [ -0.10321885]]
```

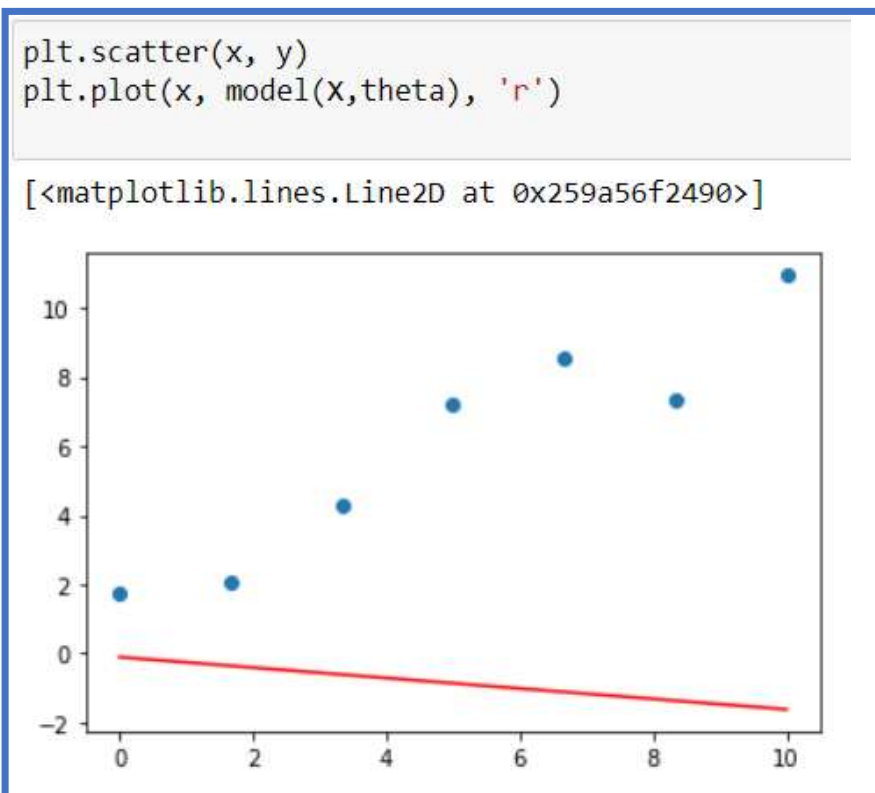
```
def model(X, theta):
    return X.dot(theta)

model(X, theta)

array([[ -0.10321885],
       [ -0.35548087],
       [ -0.60774288],
       [ -0.86000489],
       [ -1.11226691],
       [ -1.36452892],
       [ -1.61679093]])
```

Implémentation de l'algorithme de régression Linéaire

Le modèle avant l'apprentissage



Implémentation de l'algorithme de régression Linéaire

□ La Fonction Coût $J(\theta)$

la fonction coût que nous avons pour la régression linéaire est l'erreur quadratique moyenne

$$J(a, b) = \frac{1}{2m} \sum_{i=1}^m [ax^{(i)} + b - y^{(i)}]^2$$

- On sait désormais que $ax^{(i)} + b$ peut s'écrire sous la forme $X \cdot \theta$. Concernant $y^{(i)}$, nous allons créer un vecteur Y qui contient tous les $y^{(i)}$ du Dataset :

$$Y = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \dots \\ y^{(m)} \end{bmatrix}$$

Implémentation de l'algorithme de régression Linéaire

- La forme matricielle de la fonction coût est alors :

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m [X \cdot \theta - Y]^2$$

```
def function_cout(X, y, theta):  
    m = len(y)  
    return 1/(2*m) * np.sum((model(X, theta) - y)**2)
```

```
function_cout(X, y, theta)
```

```
30.44380225703108
```

Implémentation de l'algorithme de régression Linéaire

- **Les Gradients**

- Pour calculer les gradients, nous devons auparavant définir une formule spécifique pour chaque paramètre.

- $\frac{\partial J(a,b)}{\partial a} = \frac{1}{m} \sum_{i=1}^m x^{(i)} \times (ax^{(i)} + b - y^{(i)})$

- $\frac{\partial J(a,b)}{\partial b} = \frac{1}{m} \sum_{i=1}^m (ax^{(i)} + b - y^{(i)})$

- Il est possible de rassembler le calcul de ces 2 gradients en un

vecteur gradient $\frac{\partial J(\theta)}{\partial \theta} = \begin{pmatrix} \frac{\partial J(a,b)}{\partial a} \\ \frac{\partial J(a,b)}{\partial b} \end{pmatrix}$

Implémentation de l'algorithme de régression Linéaire

• Les Gradients

On a:

$$\bullet \frac{\partial J(a,b)}{\partial a} = \frac{1}{m} \sum_{i=1}^m x^{(i)} \times (ax^{(i)} + b - y^{(i)})$$

$$\bullet \frac{\partial J(a,b)}{\partial b} = \frac{1}{m} \sum_{i=1}^m (ax^{(i)} + b - y^{(i)})$$

```
def grad(X, y, theta):  
    m = len(y)  
    return 1/m * X.T.dot(model(X, theta) - y)  
  
print(grad(X, y, theta))  
  
[[ -46.22733135]  
 [ -6.89203479]]
```

$$\bullet \frac{\partial J(\theta)}{\partial \theta} = \left(\begin{array}{c} \frac{1}{m} \sum_{i=1}^m x^{(i)} \times (ax^{(i)} + b - y^{(i)}) \\ \frac{1}{m} \sum_{i=1}^m 1 \times (ax^{(i)} + b - y^{(i)}) \end{array} \right) = \frac{1}{m} X^T \cdot (X \cdot \theta - Y)$$

Implémentation de l'algorithme de régression Linéaire

$$a_{k+1} = a_k - \alpha * \frac{\partial J(a, b)}{\partial a}$$

$$b_{k+1} = b_k - \alpha * \frac{\partial J(a, b)}{\partial b}$$

$$\theta = \theta - \alpha \frac{\partial J(\theta)}{\partial \theta}$$

Définition de la méthode
d'apprentissage «
descente de gradient »

```
def gradient_descent(X, y, theta, learning_rate, n_iterations):  
    for i in range(0, n_iterations):  
        # mise a jour du parametre theta (formule du gradient descent)  
        theta = theta - learning_rate * grad(X, y, theta)  
  
    return theta
```

Implémentation de l'algorithme de régression Linéaire

Entraînement du modèle

Une fois les fonctions ci-dessus implémentées, il suffit d'utiliser la fonction **descente de gradient** en indiquant un nombre d'itérations ainsi qu'un **learning rate**, et la fonction retournera les paramètres du modèle après entraînement, sous forme de la variable **theta_final**.

```
n_iterations = 1000
learning_rate = 0.01

theta_final = gradient_descent(X, y, theta, learning_rate, n_iterations)

# voici les parametres du modele une fois que la machine a été entraînée
print(theta_final)
```

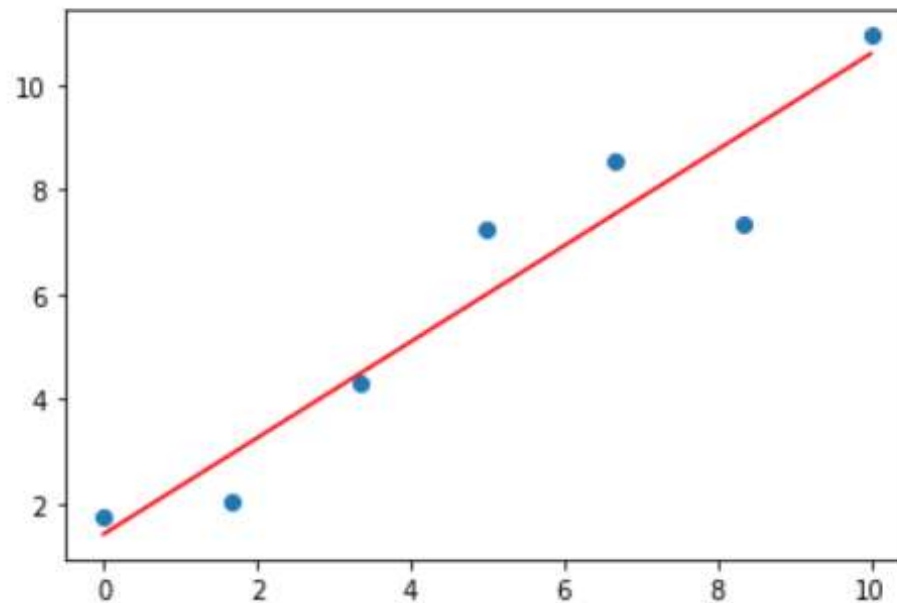
```
[[0.91734105]
 [1.42445396]]
```

```
print(cost_function(X, y, theta_final))
```

```
0.46349637203433286
```

Implémentation de l'algorithme de régression Linéaire

```
# création d'un vecteur prédictions qui contient les prédictions de notre modele final  
predictions = model(X, theta_final)  
  
# Affiche les résultats de prédictions (en rouge) par rapport a notre Dataset (en bleu)  
plt.scatter(x, y)  
plt.plot(x, predictions, c='r')  
plt.show()
```



Régression linéaire avec scikit learn



- **Scikit-learn** est une **bibliothèque libre Python** destinée à l'**apprentissage automatique**. Elle est développée par de nombreux contributeurs notamment dans le monde académique par des instituts français d'**enseignement supérieur** et de recherche comme **Inria**.
- Elle propose dans son **framework** de nombreuses bibliothèques d'**algorithmes** à implémenter.
- Ces bibliothèques sont à disposition notamment des **data scientists**.
- Elle comprend notamment des fonctions pour estimer des **forêts aléatoires**, des **régressions**, des **algorithmes de classification**, et les **machines à vecteurs de support**.

Régression linéaire avec scikit learn

Maintenant que l'on a compris le fonctionnement de la régression linéaire, voyons comment implémenter ça avec Python.

Scikit learn est la caverne d'Alibaba du **data scientist**. Quasiment tout y est ! Voici comment implémenter un modèle de régression linéaire avec scikit learn.

Pour résoudre ce problème, Nous allons récupérer des données sur **Kaggle** sur l'évolution du Ventes en fonction du nombre des TV.

On aurait séparé nos données en une **base d'entraînement** et une **base de test**.

On commence par importer les modules que l'on va utiliser :

- **Pandas** est une librairie python qui permet de manipuler facilement des données à analyser :
 - Manipuler des tableaux de données avec des étiquettes de variables (colonnes) et d'individus (lignes).
 - ces tableaux sont appelés DataFrames.
 - on peut facilement lire et écrire ces dataframes à partir ou vers un fichier.
- **Matplotlib** permet de créer des **graphiques** pour observer facilement notre dataset ainsi que le modèle construit à partir de celui-ci.

On commence par importer les modules que l'on va utiliser :

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
```

Pour trouver l'équation de régression $F(X)$, on peut utiliser le module `linear_model` de la bibliothèque d'apprentissage automatique `scikit-learn`.

On importe maintenant les données.

```
df=pd.read_csv('/content/tvmarketing.csv')  
df.shape
```

```
(200, 2)
```

```
[6] for idx,column in enumerate(df.columns):  
    print(idx,column)
```

```
0 TV
```

```
1 Sales
```

Sélectionner des colonnes de données

Extraire les données de la première colonne (TV) et la dernière (Sales)

```
X=data.iloc[:, :-1].values  
Y=data.iloc[:, -1].values
```

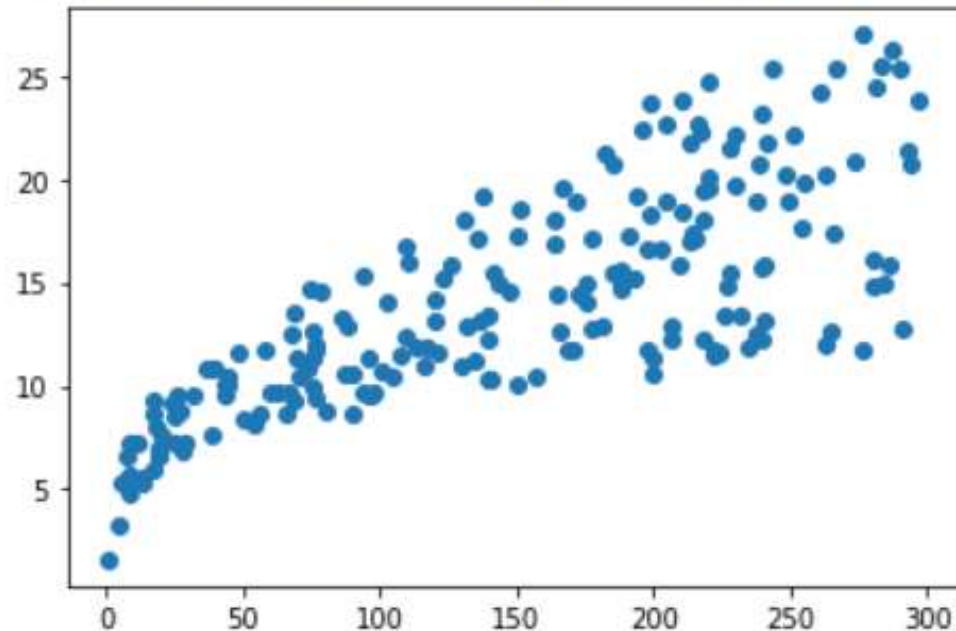
`iloc` permet Filtrer les colonnes avec des indices

on peut commencer par tracer la première variable en fonction de l'autre. On remarque bien la relation de linéarité entre les deux variables.

La fonction `plt.scatter` permet de tracer un nuage de points.

```
plt.scatter(X,Y)
```

<matplotlib.collections.PathCollection at 0x7fc102126ad0>



Séparer le **dataset** en deux :

- Dataset pour l'entraînement
- Dataset pour le test

```
X_train,X_test,Y_train,Y_test=train_test_split(X,Y,test_size=1.0/2)
```

```
len(X_train)
```

```
100
```

Le module **linear_model** de la bibliothèque **scikit-learn** nous fournit la méthode **LinearRegression()** que nous pouvons utiliser pour trouver la réponse prédite. La méthode **LinearRegression()**, lorsqu'elle est exécutée, renvoie un modèle linéaire.

```
Model_Lineaire=LinearRegression()
```

Nous pouvons former ce modèle linéaire (Entraînement du le modèle) pour trouver $F(X)$. Pour cela, nous utilisons la méthode **fit()**.

```
Model_Lineaire.fit(X_train,Y_train)
```

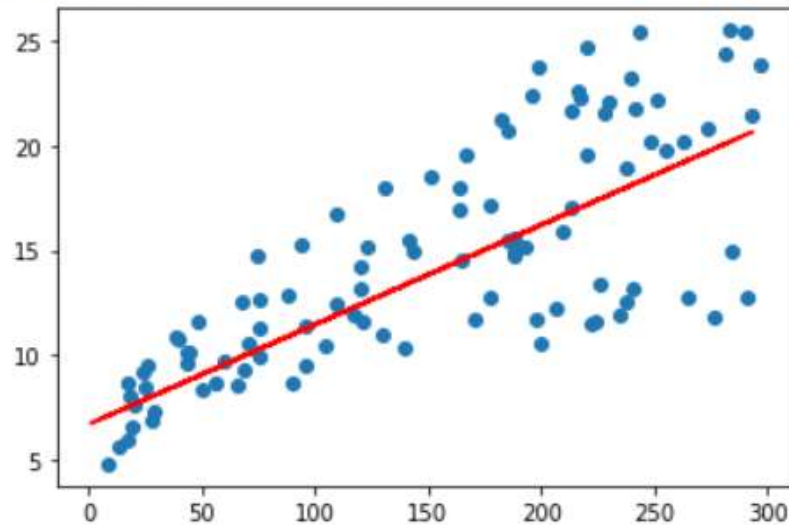
Après avoir implémenté le modèle de régression linéaire, vous pouvez prédire la valeur de Y pour tout X en utilisant la méthode **predict()**. Lorsqu'elle est invoquée sur un modèle, la méthode **predict()** prend la variable indépendante X comme argument d'entrée et renvoie la valeur prédite pour la variable dépendante Y,

```
Mod_predict=Model_Lineaire.predict(X_test)
```

Nous pouvons visualiser le modèle de régression linéaire simple à l'aide de la fonction de bibliothèque **matplotlib**. Pour cela, nous créons d'abord un nuage de points des valeurs X et Y réelles fournies en entrée. Après avoir créé le modèle de régression linéaire, nous allons tracer la sortie du modèle de régression par rapport à X en utilisant la méthode **predict()**. Cela nous donnera une ligne droite représentant le modèle de régression, comme indiqué ci-dessous.

```
plt.scatter(X_test,Y_test)  
plt.plot(X_train,Model_Lineaire.predict(X_train),color='r')
```

[<matplotlib.lines.Line2D at 0x7fc101ff8c10>]



Résumé

- Pour résoudre un problème d'apprentissage supervisé, il faut procéder en 4 étapes :

DataSet

$$X = \begin{bmatrix} x^{(1)} \\ \dots \\ x^{(m)} \end{bmatrix} \quad Y = \begin{bmatrix} y^{(1)} \\ \dots \\ y^{(m)} \end{bmatrix}$$

Modèle

$$y = ax^{(i)} + b$$

Fonction Coût

$$J(a, b) = \frac{1}{2m} \sum_{i=1}^m [ax^{(i)} + b - y^{(i)}]^2$$

Algorithme d'apprentissage

$$\frac{\partial J(a, b)}{\partial a} = \frac{1}{m} \sum_{i=1}^m x^{(i)} \times (ax^{(i)} + b - y^{(i)}) \quad a_{k+1} = a_k - \alpha * \frac{\partial J(a, b)}{\partial a}$$

$$\frac{\partial J(a, b)}{\partial b} = \frac{1}{m} \sum_{i=1}^m (ax^{(i)} + b - y^{(i)}) \quad b_{k+1} = b_k - \alpha * \frac{\partial J(a, b)}{\partial b}$$

Régression Linéaire Multiple

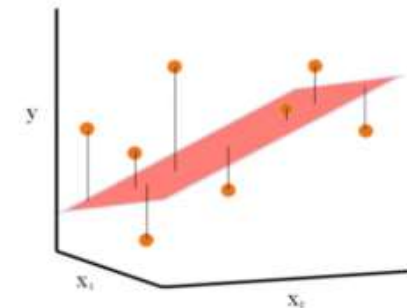
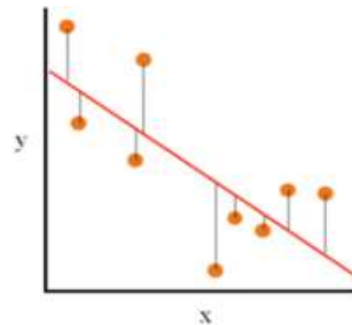
Régression Linéaire Multiple

- La **régression linéaire multiple** repose sur le même principe que la régression linéaire simple mais à part qu'elle utilise plus d'une variable explicative (dite indépendante) pour déterminer un résultat (la variable dite expliquée). Ce dernier est toujours continu alors que les variables explicatives peuvent être continues ou catégorielles. L'objectif est le même que pour la régression linéaire simple : réaliser des prédictions.
- Par exemple une régression linéaire multiple peut permettre de prédire le niveau de vente d'un produit en fonction du profil des acheteurs : âge, niveau de salaire, adresse....

Régression Linéaire Multiple

- Alors que la régression linéaire simple projette un nuage de points sur un plan graphique en deux dimensions (avec la variable explicative sur l'axe des ordonnées et la variable expliquée sur l'axe des abscisses), une régression linéaire multiple projette le nuage de points dans un graphique à plusieurs dimensions.

	Prix	Surface m2	N chambres	Qualité
€	313,000.00	124	3	1.5
€	2,384,000.00	339	5	2.5
€	342,000.00	179	3	2
€	420,000.00	186	3	2.25
€	550,000.00	180	4	2.5
€	490,000.00	82	2	1
€	335,000.00	125	2	2



Régression Linéaire Multiple

- le modèle linéaire Multiple $y = ax_1^{(i)} + bx_2^{(i)} + c$ ne permet que d'effectuer le calcul d'une prédiction à la fois, selon des valeurs $x_1^{(i)}$ et $x_2^{(i)}$.

$$X = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & 1 \\ x_1^{(2)} & x_2^{(2)} & 1 \\ \vdots & \vdots & \vdots \\ x_1^{(m)} & x_2^{(m)} & 1 \end{bmatrix}$$

$$\theta = \begin{bmatrix} a \\ b \\ c \end{bmatrix}$$

Régression Linéaire Multiple

- Pour résoudre un problème d'apprentissage supervisé, il faut procéder en 4 étapes :

DataSet

$$X = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & 1 \\ \vdots & \vdots & \vdots \\ x_1^{(m)} & x_2^{(m)} & 1 \end{bmatrix} \quad Y = \begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(m)} \end{bmatrix}$$

Modèle

$$y = ax_1^{(i)} + bx_2^{(i)} + c$$

Fonction Coût

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m [X \cdot \theta - Y]^2 \quad \theta = \begin{bmatrix} a \\ b \\ c \end{bmatrix}$$

Algorithme d'apprentissage

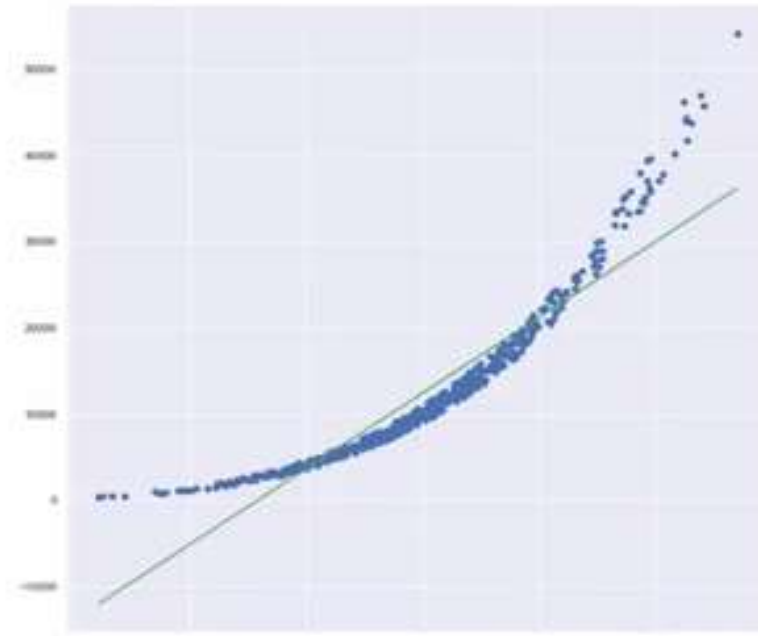
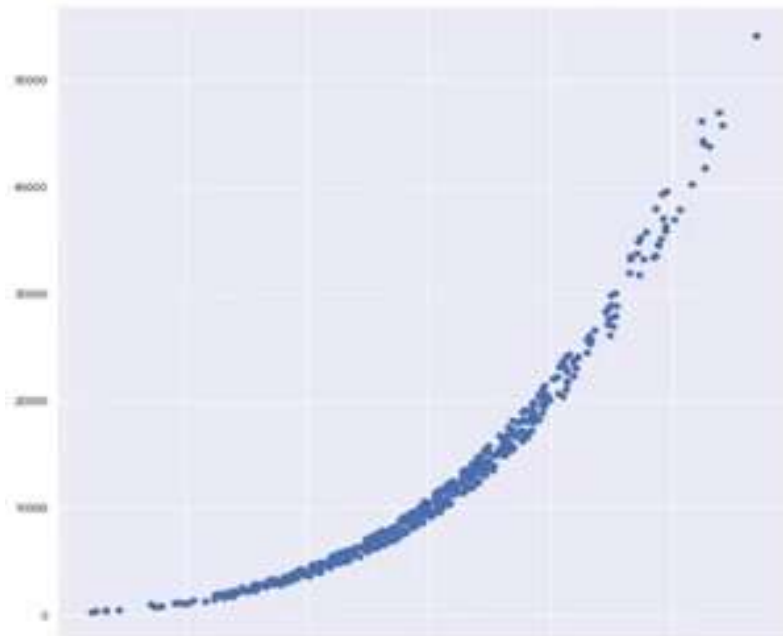
$$\theta = \theta - \alpha \frac{\partial J(\theta)}{\partial \theta} \quad \frac{\partial J(\theta)}{\partial \theta} = \frac{1}{m} X^T \cdot (X \cdot \theta - Y)$$

Régression Polynomiale

Régression Polynomiale

le modèle linéaire Multiple $y = ax^{2(i)} + bx^{(i)} + c$ ne permet que d'effectuer le calcul d'une prédiction à la fois, selon une valeur x^i unique.

$$\theta = \begin{bmatrix} a \\ b \\ c \end{bmatrix}$$



Régression Polynomiale

- Pour résoudre un problème d'apprentissage supervisé, il faut procéder en 4 étapes :

DataSet

$$X = \begin{bmatrix} x^{2(1)} & x^{(1)} & 1 \\ & \ddots & \\ x^{2(m)} & x^{(m)} & 1 \end{bmatrix} \quad Y = \begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(m)} \end{bmatrix}$$

Modèle

$$y = ax_1^{2(i)} + bx_2^{(i)} + c$$

Fonction Coût

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m [X \cdot \theta - Y]^2 \quad \theta = \begin{bmatrix} a \\ b \\ c \end{bmatrix}$$

Algorithme d'apprentissage

$$\theta = \theta - \alpha \frac{\partial J(\theta)}{\partial \theta}$$

Classification

Les Réseaux de Neurones Artificiels

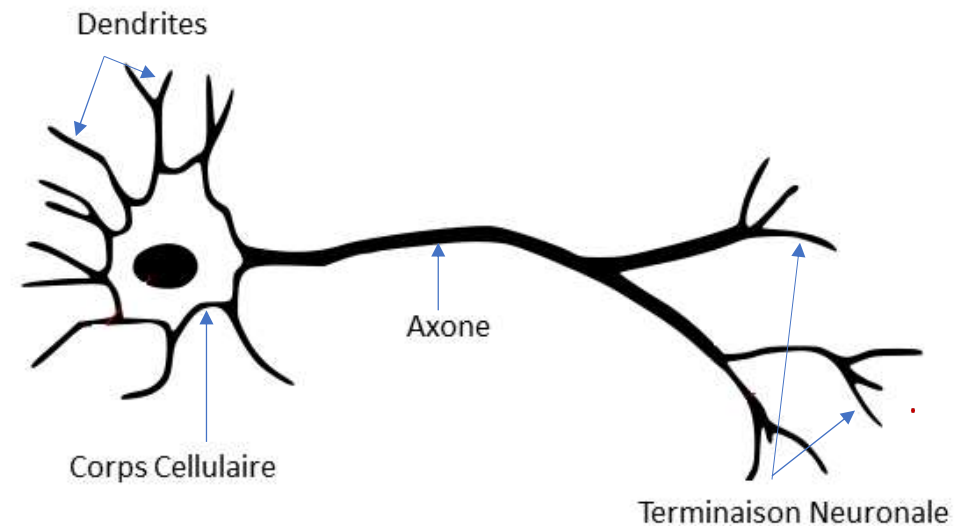
ANN

LE NEURONE BIOLOGIQUE

- Les neurones sont des cellules excitables connectées les unes aux autres et ayant pour rôle de transmettre des informations dans notre système nerveux

Le neurone biologique est constitué de trois parties :

1. Les dendrites, qui collectent les informations électriques provenant du système nerveux, ce sont les entrées du neurone ;
2. Le noyau du corps cellulaire qui traite ces informations et renvoie un signal électrique
3. L'axone, par lequel le signal sortant est transmis aux neurones voisins.

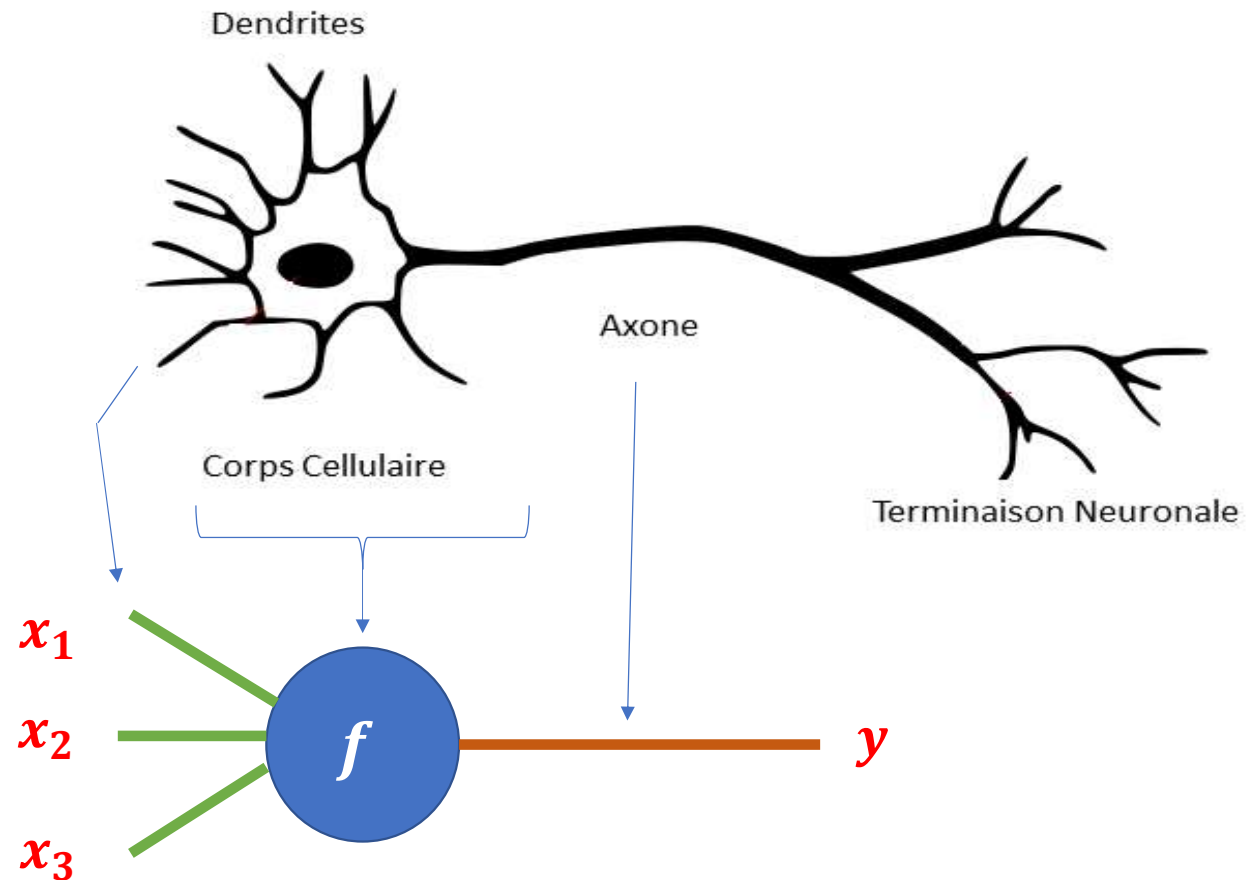


LE NEURONE FORMEL

Le neurone formel est l'élément unitaire du réseau de neurones artificiels

Le premier neurone artificiel a été inventé en 1943 par **Warren McCulloch** et **Walter Pitts** ces derniers ont modélisé le fonctionnement du neurone biologique

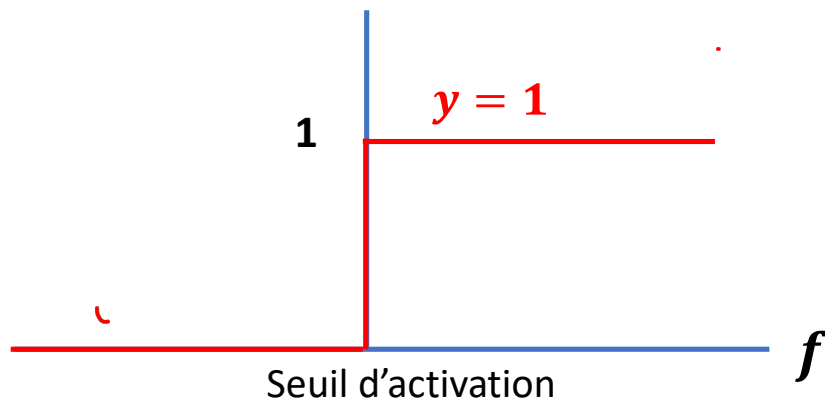
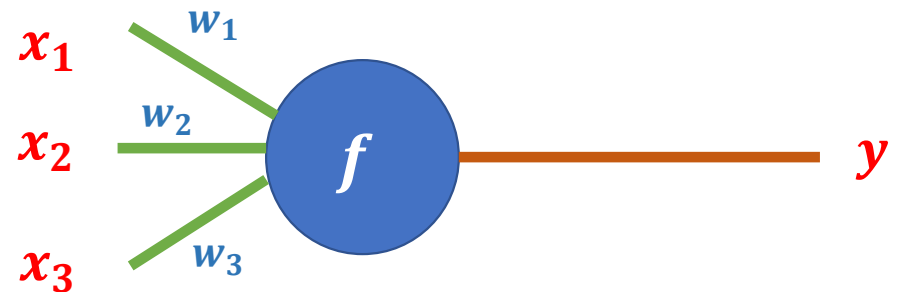
Un neurone peut être représenté par une fonction de transfert qui prend en entrée des signaux x_i et qui retourne une sortie y



LE NEURONE FORMEL

À l'intérieur de cette fonction on trouve deux étapes la première est **l'agrégation** on fait la somme des entrées de neurone en multipliant chaque entrée par un coefficient w ces coefficients représentent l'activité synaptique c-à-d le fait que le signal soit **excitateur** où w vaut **1** ou **inhibiteur** où w vaut **-1**

La deuxième étape est l'activation: si f dépasse un certain seuil en générale 0 alors le neurone s'active et retourne une sortie égale à 1 sinon reste égale à 0



➤ Agrégation $f = w_1 x_1 + w_2 x_2 + w_3 x_3$

➤ Activation
$$\begin{cases} y = 1 & \text{si } f \geq 0 \\ y = 0 & \text{sinon} \end{cases}$$

Le Perceptron

- En 1957 l'américain Frank Rosenblatt inventait le perceptron en proposant le premier algorithme d'apprentissage permettant de trouver les poids w afin de trouver les sorties y désirées
- Entraîner un neurone artificiel sur des données de références (X,y) pour que celui-ci renforce ses paramètres w .

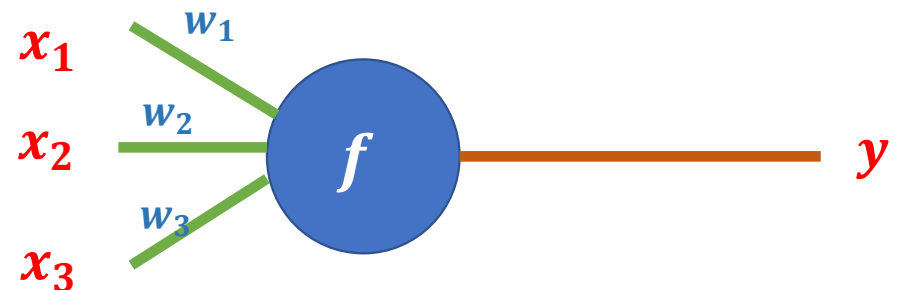
$$w_i = w_i + \alpha(Y_{ref} - y_{neur})X_i$$

Y_{ref} sortie de référence

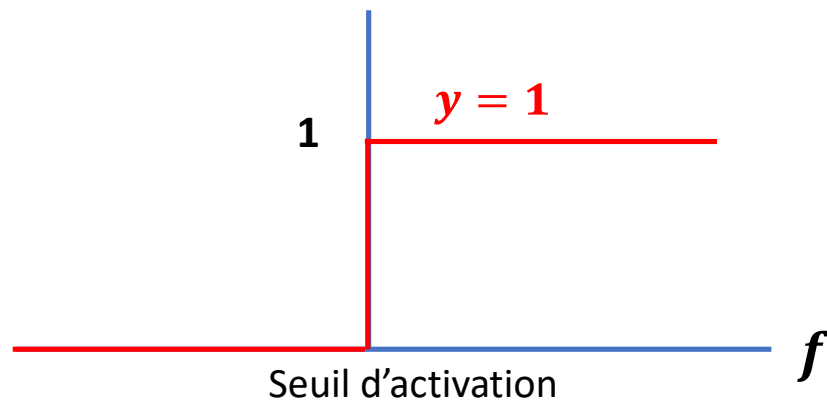
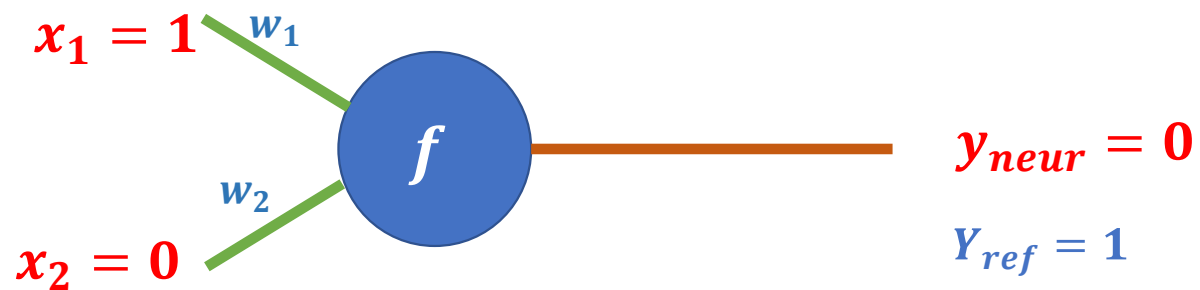
y_{neur} sortie produit par le neurone

X_i entrée du neurone

α Vitesse d'apprentissage



Le Perceptron



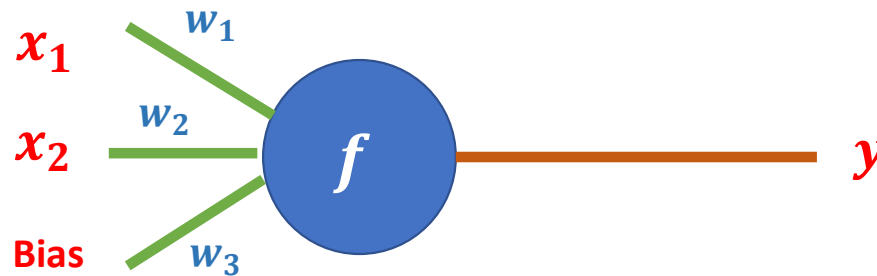
$$w_i = w_i + \alpha(Y_{ref} - y_{neur})X_i$$

$$w_i = w_i + \alpha(1 - 0)X_i$$

$$w_i = w_i + \alpha X_i$$

Exemple

x_1	x_2	d
1.0	1.0	1
9.4	6.4	-1
2.5	2.1	1
8.0	7.7	-1
0.5	2.2	1
7.9	8.4	-1
7.0	7.0	-1
2.8	0.8	1
1.2	3.0	1
7.8	6.1	-1



Paramètres

x_i : entrée

w_i : poids (nombre réel)

c : pas d'apprentissage: $0 \leq c \leq 1$

d : sortie désirée

1. Initialisation : $w = [w_1, w_2, w_3] = [.75, .5, -.6]$

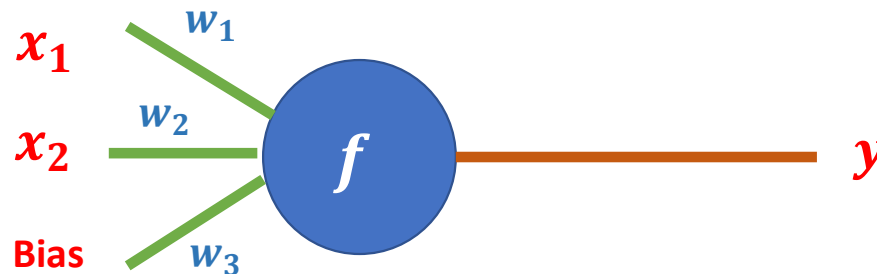
Algorithme d'apprentissage

Pour chaque donnée d'entraînement,
incrémenter le poids w_i par :

$$\Delta w_i = c(d-y)x_i$$

Exemple

x_1	x_2	d
1.0	1.0	1
9.4	6.4	-1
2.5	2.1	1
8.0	7.7	-1
0.5	2.2	1
7.9	8.4	-1
7.0	7.0	-1
2.8	0.8	1
1.2	3.0	1
7.8	6.1	-1



Paramètres

x_i : entrée

w_i : poids (nombre réel)

c : pas d'apprentissage: $0 \leq c \leq 1$

d : sortie désirée

Algorithme d'apprentissage

Pour chaque donnée d'entraînement, incrémenter le poids w_i par :

$$\Delta w_i = c(d-y)x_i$$

1. Initialisation : $w = [w_1, w_2, w_3] = [.75, .5, -.6]$

2. $f(\text{net}) = \text{sign}(.75 \times 1 + .5 \times 1 - .6 \times 1) = \text{sign}(.65) = 1;$

$\Delta w = 0.2(1-1)X = 0$; donc w est inchangé.

3. $f(\text{net}) = \text{sign}(.75 \times 9.4 + .5 \times 6.4 - .6 \times 1) = \text{sign}(9.65) = 1;$

$\Delta w = -.4X$; donc $w = w - .4 [9.4, 6.4, 1] = [-3.01, -2.06, -1]$

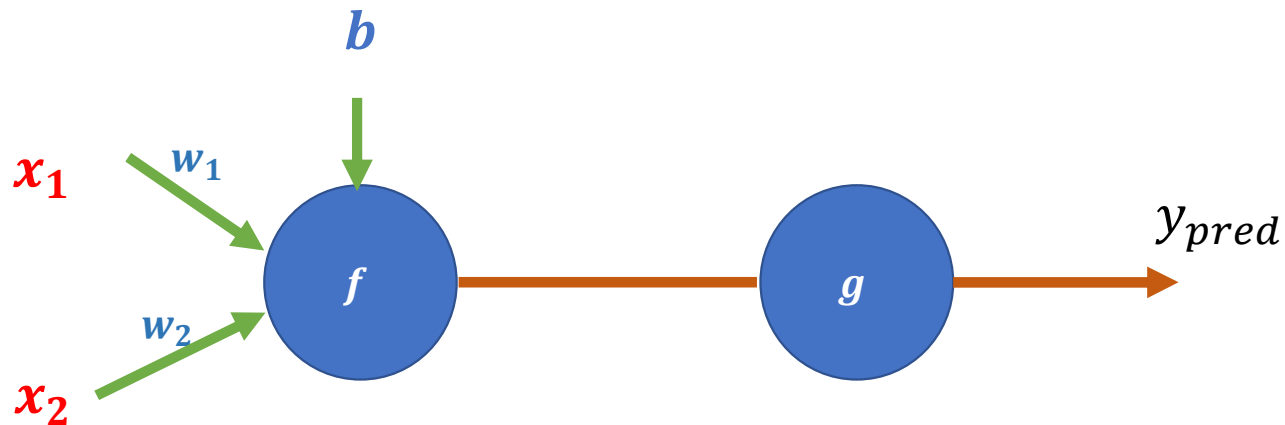
...

500. $w = [-1.3, -1.1, +10.9].$

Équation de la ligne séparant les données : $-1.3x_1 + -1.1x_2 + 10.9 = 0$

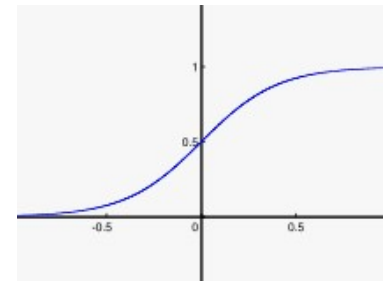
Le Perceptron

Le Modèle



$$f = w_1 x_1 + w_2 x_2 + b$$

$$g = \frac{1}{1 + e^{-f}}$$



La fonction d'activation est la fonction sigmoïde

Le Perceptron

Il existe plusieurs fonctions d'activation, on peut noter les plus utilisées :

Pas unitaire	$g(x) = \begin{cases} 0 & \text{si } x < 0 \\ 1 & \text{si } x \geq 0 \end{cases}$
Sigmoïde	$g(x) = \frac{1}{1 + e^{-x}}$
Linéaire avec seuil	$g(x) = \begin{cases} 0 & \text{si } x < x_{min} \\ mx + b & \text{si } x_{min} < x < x_{max} \\ 1 & \text{si } x \geq x_{max} \end{cases}$
Gaussien	$g(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{\frac{-(x-\mu)^2}{2\sigma^2}}$
Identité	$g(x) = x$

Le Perceptron

- Dans le cas d'un problème de **régression**, il n'est pas nécessaire de transformer la somme pondérée reçue en entrée. La fonction d'activation est la fonction identité, elle retourne ce qu'elle a reçu en entier.
- Dans le cas d'un problème de **classification binaire**, on peut utiliser une fonction de **seuil** :
$$\begin{cases} y = 1 & \text{si } f \geq 0 \\ y = 0 & \text{sinon} \end{cases}$$
- on peut aussi utiliser une fonction **sigmoïde** pour prédire la *probabilité* d'appartenir à la classe positive :
$$g(f) = \frac{1}{1+e^{-f}}$$

Le Perceptron

Objectif

- Régler les paramètres w_i et b pour obtenir le meilleur modèle qui fait les plus petites erreurs entre les sorties du neurone $g(f)$ et les vraies données de sorties y et donc on définit la fonction coût qui définit les erreurs

Le Perceptron

La fonction Coût

Dans le cas de la **régression**, nous avons choisi l'**Erreur Quadratique Moyenne** (comme pour une régression linéaire) : $E = \frac{1}{2m} \sum_{i=1}^m [g^{(i)} - y^{(i)}]^2$

Dans le cas de la **classification**, nous allons choisir la fonction perte de log (log Loss) qui est définie par:

$$E = -\frac{1}{m} \sum_{i=1}^m y^{(i)} \log(g^{(i)}) + (1 - y^{(i)}) \log(1 - g^{(i)})$$

Le Perceptron

Apprentissage

- Le perceptron est entraîné par des **mises à jour itératives** de ses poids grâce à **l'algorithme du gradient**. La même règle de mise à jour des poids s'applique dans le cas de la régression, de la classification binaire ou de la classification multi-classe

Le Perceptron

Apprentissage

- L'entraînement d'un perceptron est donc un processus **itératif**. Après chaque observation, nous allons ajuster les poids de connexion de sorte à réduire l'erreur de prédiction faite par le perceptron dans son état actuel. Pour cela, nous allons utiliser [l'algorithme de descente du gradient](#) : le gradient nous donnant la direction de plus grande variation d'une fonction (dans notre cas, la fonction d'erreur), pour trouver le minimum de cette fonction il faut se déplacer dans la direction opposée au gradient. (Lorsque la fonction est minimisée localement, son gradient est égal à 0.)

Le Perceptron

Apprentissage

- La règle de mise à jour est :

$$w_{t+1} = w_t - \alpha \frac{\partial E}{\partial w_t}$$

w_t : Le poids à l'instant t

w_{t+1} : Le poids à l'instant t+1

α : Le pas d'apprentissage

$\frac{\partial E}{\partial w_t}$: Gradient à l'instant t

$$\begin{aligned}w_1 &= w_1 - \alpha \frac{\partial E}{\partial w_1} \\w_2 &= w_2 - \alpha \frac{\partial E}{\partial w_2} \\b &= b - \alpha \frac{\partial E}{\partial b}\end{aligned}$$



$$\begin{aligned}\frac{\partial E}{\partial w_1} &= \frac{1}{m} \sum_{i=1}^m (g^{(i)} - y^{(i)}) x_1 \\ \frac{\partial E}{\partial w_2} &= \frac{1}{m} \sum_{i=1}^m (g^{(i)} - y^{(i)}) x_2 \\ \frac{\partial E}{\partial b} &= \frac{1}{m} \sum_{i=1}^m (g^{(i)} - y^{(i)})\end{aligned}$$

Classification

- Pour résoudre un problème d'apprentissage supervisé, il faut procéder en 4 étapes :

DataSet

$$X = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} \\ \vdots & \vdots \\ x_1^{(m)} & x_2^{(m)} \end{bmatrix} \quad Y = \begin{bmatrix} y^{(1)} \\ \dots \\ y^{(m)} \end{bmatrix}$$

Modèle

$$f = w_1 x_1 + w_2 x_2 + b \quad g = \frac{1}{1+e^{-f}}$$

Fonction Coût

$$E = -\frac{1}{m} \sum_{i=1}^m y^{(i)} \log(g^{(i)}) + (1 - y^{(i)}) \log(1 - g^{(i)})$$

Algorithme d'apprentissage

$$\begin{aligned} w_1 &= w_1 - \alpha \frac{\partial E}{\partial w_1} \\ w_2 &= w_2 - \alpha \frac{\partial E}{\partial w_2} \\ b &= b - \alpha \frac{\partial E}{\partial b} \end{aligned}$$



$$\begin{aligned} \frac{\partial E}{\partial w_1} &= \frac{1}{m} \sum_{i=1}^m (g^{(i)} - y^{(i)}) x_1 \\ \frac{\partial E}{\partial w_2} &= \frac{1}{m} \sum_{i=1}^m (g^{(i)} - y^{(i)}) x_2 \\ \frac{\partial E}{\partial b} &= \frac{1}{m} \sum_{i=1}^m (g^{(i)} - y^{(i)}) \end{aligned}$$

Implémentation